

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT HUNG YÊN



BÀI TẬP LỚN

MÔN: HỌC MÁY CƠ BẢN

**Dự đoán nhân viên nghỉ việc bằng
Logistic Regression, Random Forest**

NGÀNH: KHOA HỌC MÁY TÍNH
CHUYÊN NGÀNH: TRÍ TUỆ NHÂN TẠO VÀ KHOA HỌC DỮ LIỆU

SINH VIÊN: Nguyễn Hữu Quang

MÃ SINH VIÊN: 12423066

MÃ LỚP: 12423TN

GIẢNG VIÊN HƯỚNG DẪN: PGS.TS NGUYỄN VĂN HẬU

HUNG YÊN – 2025

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT HUNG YÊN



BÀI TẬP LỚN

MÔN: HỌC MÁY CƠ BẢN

**Dự đoán nhân viên nghỉ việc bằng
Logistic Regression, Random Forest**

NGÀNH: KHOA HỌC MÁY TÍNH
CHUYÊN NGÀNH: TRÍ TUỆ NHÂN TẠO VÀ KHOA HỌC DỮ LIỆU

SINH VIÊN: Nguyễn Hữu Quang

MÃ SINH VIÊN: 12423066

MÃ LỚP: 12423TN

GIẢNG VIÊN HƯỚNG DẪN: PGS.TS NGUYỄN VĂN HẬU

HUNG YÊN – 2025

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

Ký và ghi họ tên

LỜI CAM ĐOAN

Em xin cam đoan bài tập lớn môn Học máy cơ bản là kết quả thực hiện của chúng em dưới sự hướng dẫn của thầy Nguyễn Văn Hậu

Những phần sử dụng tài liệu tham khảo trong bài tập lớn đã được nêu rõ trong phần tài liệu tham khảo. Các kết quả trình bày trong bài tập lớn và chương trình xây dựng được hoàn toàn là kết quả do bản thân em thực hiện.

Nếu vi phạm lời cam đoan này, em xin chịu hoàn toàn trách nhiệm trước khoa và nhà trường.

Hưng yên, ngày 28 tháng 12 năm 2025

Sinh viên

Nguyễn Hữu Quang

LỜI CẢM ƠN

Để có thể hoàn thành bài tập lớn này, lời đầu tiên em xin phép gửi lời cảm ơn tới bộ môn Khoa học máy tính, Khoa Công nghệ thông tin – Trường Đại học Sư phạm Kỹ thuật Hưng Yên đã tạo điều kiện thuận lợi cho em thực hiện bài tập lớn môn học này.

Đặc biệt em xin chân thành cảm ơn thầy Nguyễn Văn Hậu đã rất tận tình hướng dẫn, chỉ bảo em trong suốt thời gian thực hiện bài tập lớn vừa qua.

Mặc dù chúng em đã có cố gắng, nhưng với trình độ còn hạn chế, trong quá trình thực hiện không tránh khỏi những thiếu sót. Em hy vọng sẽ nhận được những ý kiến nhận xét, góp ý của cô về những kết quả triển khai trong bài tập lớn.

Em xin trân trọng cảm ơn!

MỤC LỤC

MỤC LỤC.....	5
CHƯƠNG 1: TỔNG QUAN BÀI TOÁN	9
1.1 Bài toán	9
1.2 Trình bày dữ liệu bài toán.....	9
1.3 Tiền xử lý dữ liệu.....	13
1.4 Trực quan dữ liệu	14
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	15
2.1 Pandas	15
2.1.1 Giới thiệu về Pandas	15
2.1.2 Cài đặt Pandas.....	15
2.2 Numpy.....	16
2.2.1 Giới thiệu về Numpy	16
2.2.2 Cài đặt Numpy	16
2.3 Matplotlib.....	16
2.3.1 Giới thiệu về Matplotlib	16
2.3.2 Cài đặt Matplotlib	17
2.4 Random Forset	17
2.4.1 Cấu trúc của Random Forest.....	17
2.4.2 Cách thức hoạt động của Random Forest	18
2.4.3 Ứng dụng của Random Forest	18
2.5 Logistic Regression.....	19
2.5.1 Cấu trúc của Logistic Regression	19

2.5.2	Cách thức hoạt động của Logistic Regression.....	19
2.5.3	Ứng dụng của Logistic Regression.....	20
2.5.4	Loss Function.....	20
2.5.5	Các loại Logistic Regression	20
CHƯƠNG 3: NỘI DUNG THỰC HIỆN VÀ KẾT QUẢ		21
3.1	Mã nguồn tiền xử lý dữ liệu.....	21
3.2	Mã nguồn thống kê dữ liệu	23
3.3	Mã nguồn trực quan hóa	27
3.4	Áp dụng Model	42
3.4.1	Model Logistic Regression	48
3.4.2	Model Random Forest	51
3.4.3	So sánh và đánh giá mô hình	53
3.5	Sản phẩm.....	54
KẾT LUẬN.....		56
TÀI LIỆU THAM KHẢO.....		57

DANH MỤC HÌNH ẢNH

Hình 1.1: Dữ liệu bộ Dataset WA_Fn-UseC_-HR-Employee-Attrition.....	10
Hình 1.2: Thông tin từng cột của Dataset	10
Hình 1.3: Mô tả dữ liệu từng cột của Dataset	13
Hình 3.1: Thông tin từng cột của Dataset	21
Hình 3.2: Dữ liệu sau khi xóa cột dư thừa	22
Hình 3.3: Kiểm tra giá trị trùng lặp.....	22
Hình 3.4: Kiểm tra giá trị thiếu	23
Hình 3.5: Mã nguồn đếm số lượng giới tính.....	24
Hình 3.6: Mã nguồn đếm số lượng nhân viên nghỉ việc	24
Hình 3.7: Mã nguồn đếm số lượng tình trạng hôn nhân của nhân viên.....	24
Hình 3.8: Mã nguồn đếm số lượng nhân viên có lương hàng tháng cao hơn mức trung bình	25
Hình 3.9: Mã nguồn đếm số lượng vị trí công việc của nhân viên	25
Hình 3.10: Mã nguồn đếm số lượng cấp bậc công việc của nhân viên	26
Hình 3.11: Mã nguồn đếm số lượng nhân viên có khoảng cách từ nhà đến công ty xa hơn mức trung bình	26
Hình 3.12: Mã nguồn biểu đồ phân phối tỉ lệ nhân viên nghỉ việc.....	27
Hình 3.13: Biểu đồ phân phối tỉ lệ nhân viên nghỉ việc.....	28
Hình 3.14: Mã nguồn biểu đồ heatmap.....	28
Hình 3.15: Biểu đồ tương quan.....	30
Hình 3.16: Biểu đồ phân phối tuổi theo trạng thái nghỉ việc	30
Hình 3.17: Biểu đồ phân phối tuổi theo trạng thái nghỉ việc	31
Hình 3.18: Mã nguồn biểu đồ phân phối thu nhập theo trạng thái nghỉ việc.....	32
Hình 3.19: Biểu đồ phân phối thu nhập theo trạng thái nghỉ việc	33
Hình 3.20: Mã nguồn biểu đồ phân phối khoảng cách từ nhà đến công ty theo trạng thái nghỉ việc	33
Hình 3.21: Biểu đồ phân phối khoảng cách từ nhà đến công ty theo trạng thái nghỉ việc	34
Hình 3.22: Mã nguồn biểu đồ scatterplot mối quan hệ giữa tuổi, thu nhập và nghỉ việc	35

Hình 3.23: Biểu đồ phân tán mối quan hệ giữa tuổi, thu nhập và nghỉ việc.....	36
Hình 3.24: Mã nguồn biểu đồ phân bố lĩnh vực giáo dục theo trạng thái nghỉ việc.....	36
Hình 3.25: Biểu đồ phân bố lĩnh vực giáo dục theo trạng thái nghỉ việc	37
Hình 3.26: Mã nguồn biểu đồ phân bố tình trạng hôn nhân theo trạng thái nghỉ việc	38
Hình 3.27: Biểu đồ phân bố tình trạng hôn nhân theo trạng thái nghỉ việc	39
Hình 3.28: Mã nguồn biểu đồ số năm làm việc tại công ty theo trạng thái nghỉ việc	39
Hình 3.29: Biểu đồ số năm làm việc tại công ty theo trạng thái nghỉ việc	40
Hình 3.30: Mã nguồn biểu đồ boxplot cho từng cột theo trạng thái nghỉ việc	41
Hình 3.31: Biểu đồ boxplot cho từng cột theo trạng thái nghỉ việc	42
Hình 3.32: Chuẩn hóa dữ liệu	43
Hình 3.33: Mã hóa dữ liệu	45
Hình 3.34: Mã nguồn chia 2 tập train và test	46
Hình 3.35: Kết quả sau khi chia dữ liệu.....	46
Hình 3.36: Xử lý mất cân bằng dữ liệu bằng SMOTE.....	47
Hình 3.37: Mã nguồn huấn luyện mô hình	47
Hình 3.38: Model logistic regression trước smote	49
Hình 3.39: Biểu đồ Precision-Recall, ROC và Predicted lable của model logistic regression trước smote	50
Hình 3.40: Model logistic regression sau smote	50
Hình 3.41: Biểu đồ Precision-Recall, ROC và Predicted lable của model logistic regression sau smote	51
Hình 3.42: Model random forest.....	51
Hình 3.43: Biểu đồ Precision-Recall, ROC và Predicted lable của model random forest.....	52
Hình 3.44: So sánh hai mô hình	53
Hình 3.45: Tiêu đề sản phẩm	54
Hình 3.46: Giao diện sản phẩm.....	54
Hình 3.47: Kết quả dự đoán bằng model logistic regression	55

CHƯƠNG 1: TỔNG QUAN BÀI TOÁN

1.1 Bài toán

Lĩnh vực quản lý nhân sự kỹ thuật số, đặc biệt là việc ứng dụng học máy trong dự đoán nhân viên nghỉ việc, đang trở thành một trong những giải pháp quan trọng nhằm nâng cao hiệu quả tổ chức và giảm thiểu rủi ro kinh doanh. Bằng cách phân tích dữ liệu nhân viên và các chỉ số hành vi, chúng ta có thể thu thập thông tin chi tiết về các yếu tố nguy cơ, xu hướng luân chuyển nhân sự và khả năng nghỉ việc của từng cá nhân. Dữ liệu này thường được tổng hợp từ các hồ sơ nhân sự, các cuộc khảo sát nội bộ và các chỉ số đánh giá hiệu suất uy tín.

Việc phân tích dữ liệu này không chỉ giúp doanh nghiệp chủ động hơn trong việc giữ chân nhân tài mà còn cung cấp cái nhìn tổng quan về tác động của các yếu tố như mức lương, mức độ hài lòng công việc, thời gian làm thêm và môi trường làm việc đến nguy cơ nghỉ việc. Các mô hình dự đoán được cập nhật liên tục dựa trên dữ liệu thực tế, đảm bảo tính chính xác và độ tin cậy, giúp phân loại mức độ rủi ro từ ổn định đến cảnh báo cao.

Ngoài ra, việc dự báo sớm nguy cơ nghỉ việc có ảnh hưởng lớn đến chiến lược quản lý nguồn nhân lực của doanh nghiệp. Việc dự đoán chính xác không chỉ giúp nhân viên có kế hoạch phát triển sự nghiệp kịp thời mà còn hỗ trợ lãnh đạo trong việc tối ưu hóa chính sách nhân sự và phân bổ nguồn lực. Thông qua các công cụ phân tích dữ liệu, doanh nghiệp có thể đưa ra quyết định giữ chân nhân tài phù hợp, từ đó giảm thiểu chi phí tuyển dụng và tìm được hướng phát triển tổ chức tối ưu nhất cho tương lai.

1.2 Trình bày dữ liệu bài toán

Dữ liệu bài toán được chọn lọc và lấy từ kaggle có đường địa chỉ như sau: <https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset> có dạng csv với tên gọi: WA_Fn-UseC_-HR-Employee-Attrition.

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Life Sciences	1
1	49	No	Travel_Frequently	279	Research & Development	8	1	Life Sciences	1
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	Other	1
3	33	No	Travel_Frequently	1392	Research & Development	3	4	Life Sciences	1
4	27	No	Travel_Rarely	591	Research & Development	2	1	Medical	1
...	...	...	...	...	...	...	...	...	...
1465	36	No	Travel_Frequently	884	Research & Development	23	2	Medical	1
1466	39	No	Travel_Rarely	613	Research & Development	6	1	Medical	1
1467	27	No	Travel_Rarely	155	Research & Development	4	3	Life Sciences	1
1468	49	No	Travel_Frequently	1023	Sales	2	3	Medical	1
1469	34	No	Travel_Rarely	628	Research & Development	8	3	Medical	1

1470 rows × 35 columns

Hình 1.1: Dữ liệu bộ Dataset WA_Fn-UseC_-HR-Employee-Attrition

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 35 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Age                                    1470 non-null   int64
1   Attrition                             1470 non-null   object
2   BusinessTravel                         1470 non-null   object
3   DailyRate                             1470 non-null   int64
4   Department                             1470 non-null   object
5   DistanceFromHome                       1470 non-null   int64
6   Education                              1470 non-null   int64
7   EducationField                         1470 non-null   object
8   EmployeeCount                          1470 non-null   int64
9   EmployeeNumber                         1470 non-null   int64
10  EnvironmentSatisfaction                 1470 non-null   int64
11  Gender                                 1470 non-null   object
12  HourlyRate                             1470 non-null   int64
13  JobInvolvement                         1470 non-null   int64
14  JobLevel                               1470 non-null   int64
15  JobRole                                1470 non-null   object
16  JobSatisfaction                        1470 non-null   int64
17  MaritalStatus                          1470 non-null   object
18  MonthlyIncome                          1470 non-null   int64
19  MonthlyRate                            1470 non-null   int64
20  NumCompaniesWorked                     1470 non-null   int64
21  Over18                                 1470 non-null   object
22  OverTime                               1470 non-null   object
23  PercentSalaryHike                      1470 non-null   int64
24  PerformanceRating                      1470 non-null   int64
25  RelationshipSatisfaction                1470 non-null   int64
26  StandardHours                          1470 non-null   int64
27  StockOptionLevel                       1470 non-null   int64
28  TotalWorkingYears                      1470 non-null   int64
```

Hình 1.2: Thông tin từng cột của Dataset

- Dữ liệu bài toán gồm các cột:
 - Age: Tuổi của nhân viên
 - Attrition: Nhân viên đã nghỉ việc hay chưa (Yes/No)

- BusinessTravel: Tần suất đi công tác (ví dụ: Hiếm khi - Rarely, Thường xuyên - Frequently)
- DailyRate: Mức lương theo ngày của nhân viên
- Department: Phòng ban nơi nhân viên làm việc
- DistanceFromHome: Khoảng cách từ nhà nhân viên đến nơi làm việc (tính bằng km hoặc dặm)
- Education: Trình độ học vấn (được mã hóa bằng số, ví dụ: 1 = Dưới Cao đẳng, 2 = Cao đẳng)
- EducationField: Ngành học (ví dụ: Khoa học Đời sống, Marketing)
- EmployeeCount: Số lượng nhân viên (giá trị hằng số trong bộ dữ liệu này)
- EmployeeNumber: Mã định danh duy nhất cho từng nhân viên
- EnvironmentSatisfaction: Mức độ hài lòng với môi trường làm việc (thang điểm 1–4)
- Gender: Giới tính của nhân viên (Nam/Nữ)
- HourlyRate: Mức lương theo giờ của nhân viên
- JobInvolvement: Mức độ tham gia/gắn bó với công việc (thang điểm 1–4)
- JobLevel: Cấp bậc công việc trong công ty
- JobRole: Vai trò hoặc chức danh cụ thể của nhân viên
- JobSatisfaction: Đánh giá mức độ hài lòng với công việc (thang điểm 1–4)
- MaritalStatus: Tình trạng hôn nhân của nhân viên
- MonthlyIncome: Thu nhập hàng tháng của nhân viên
- MonthlyRate: Mức trả lương hàng tháng
- NumCompaniesWorked: Số lượng công ty nhân viên đã từng làm việc trước đây
- Over18: Nhân viên có trên 18 tuổi hay không (Yes/No)
- OverTime: Nhân viên có làm thêm giờ hay không (Yes/No)
- PercentSalaryHike: Phần trăm tăng lương
- PerformanceRating: Đánh giá hiệu suất làm việc (thang điểm 1–4)
- RelationshipSatisfaction: Mức độ hài lòng với các mối quan hệ tại nơi làm việc (thang điểm 1–4)

- StandardHours: Giờ làm việc tiêu chuẩn (giá trị hằng số trong bộ dữ liệu)
- StockOptionLevel: Mức độ quyền chọn mua cổ phiếu (cấp độ 0–3)
- TotalWorkingYears: Tổng số năm kinh nghiệm làm việc chuyên nghiệp
- TrainingTimesLastYear: Số lần tham gia đào tạo trong năm ngoái
- WorkLifeBalance: Đánh giá cân bằng giữa công việc và cuộc sống (thang điểm 1–4)
- YearsAtCompany: Tổng số năm làm việc tại công ty hiện tại
- YearsInCurrentRole: Số năm đảm nhiệm vai trò/vị trí hiện tại
- YearsSinceLastPromotion: Số năm kể từ lần thăng chức cuối cùng
- YearsWithCurrManager: Số năm làm việc dưới quyền người quản lý hiện tại
- Dữ liệu bài toán là một file csv gồm: 1470 rows x 35 columns
 - File csv: WA_Fn-UseC_-HR-Employee-Attrition
 - Có 35 cột và 1470 bản ghi
- Dữ liệu sau khi mô tả, ta có :

```
df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
Age	1470.0	36.923810	9.135373	18.0	30.00	36.0	43.00	60.0
DailyRate	1470.0	802.485714	403.509100	102.0	465.00	802.0	1157.00	1499.0
DistanceFromHome	1470.0	9.192517	8.106864	1.0	2.00	7.0	14.00	29.0
Education	1470.0	2.912925	1.024165	1.0	2.00	3.0	4.00	5.0
EmployeeCount	1470.0	1.000000	0.000000	1.0	1.00	1.0	1.00	1.0
EmployeeNumber	1470.0	1024.865306	602.024335	1.0	491.25	1020.5	1555.75	2068.0
EnvironmentSatisfaction	1470.0	2.721769	1.093082	1.0	2.00	3.0	4.00	4.0
HourlyRate	1470.0	65.891156	20.329428	30.0	48.00	66.0	83.75	100.0
JobInvolvement	1470.0	2.729932	0.711561	1.0	2.00	3.0	3.00	4.0
JobLevel	1470.0	2.063946	1.106940	1.0	1.00	2.0	3.00	5.0
JobSatisfaction	1470.0	2.728571	1.102846	1.0	2.00	3.0	4.00	4.0
MonthlyIncome	1470.0	6502.931293	4707.956783	1009.0	2911.00	4919.0	8379.00	19999.0
MonthlyRate	1470.0	14313.103401	7117.786044	2094.0	8047.00	14235.5	20461.50	26999.0
NumCompaniesWorked	1470.0	2.693197	2.498009	0.0	1.00	2.0	4.00	9.0
PercentSalaryHike	1470.0	15.209524	3.659938	11.0	12.00	14.0	18.00	25.0
PerformanceRating	1470.0	3.153741	0.360824	3.0	3.00	3.0	3.00	4.0
RelationshipSatisfaction	1470.0	2.712245	1.081209	1.0	2.00	3.0	4.00	4.0
StandardHours	1470.0	80.000000	0.000000	80.0	80.00	80.0	80.00	80.0

Hình 1.3: Mô tả dữ liệu từng cột của Dataset

- Trong Dataset cần chuẩn hóa dữ liệu đầu vào và xử lý bất thường dữ liệu.

1.3 Tiền xử lý dữ liệu

1. Xử lý thông tin các cột, Gender (giới tính), Attrition(rời bỏ)
2. Loại bỏ cột dư thừa
3. Chuẩn hóa dữ liệu các cột : Age, DailyRate, DistanceFromHome, HourlyRate, MonthlyIncome, MonthlyRate, TotalWorkingYears, PercentSalaryHike, YearsAtCompany, YearsInCurrentRole, YearsSinceLastPromotion, YearsWithCurrManager, NumCompaniesWorked, TrainingTimesLastYear.

4. Mã hóa dữ liệu cột: Education, EnvironmentSatisfaction, JobInvolvement, JobSatisfaction, PerformanceRating, RelationshipSatisfaction, WorkLifeBalance, StockOptionLevel, JobLevel, BusinessTravel, Department, EducationField, JobRole, MaritalStatus, Gender, OverTime.

1.4 Trực quan dữ liệu

1. Biểu đồ phân phối tỉ lệ nhân viên nghỉ việc
2. Biểu đồ tương quan
3. Biểu đồ phân phối tuổi theo trạng thái nghỉ việc
4. Biểu đồ phân phối thu nhập theo trạng thái nghỉ việc
5. Biểu đồ phân phối khoảng cách từ nhà đến công ty theo trạng thái nghỉ việc
6. Biểu đồ figure mối quan hệ giữa tuổi, thu nhập và nghỉ việc
7. Biểu đồ phân bố lĩnh vực giáo dục theo trạng thái nghỉ việc
8. Biểu đồ phân bố tình trạng hôn nhân theo trạng thái nghỉ việc
9. Biểu đồ phân bố số năm làm việc tại công ty theo trạng thái nghỉ việc
10. Biểu đồ Boxplot cho từng cột theo trạng thái nghỉ việc

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Pandas

2.1.1 Giới thiệu về Pandas

Pandas là một thư viện mã nguồn mở phổ biến trong Python, được xây dựng trên nền tảng NumPy, chuyên dùng để thao tác và phân tích dữ liệu. Với khả năng xử lý các loại dữ liệu đa dạng như chuỗi thời gian, bảng không đồng nhất và ma trận dữ liệu, Pandas mang lại trải nghiệm làm việc trực quan và linh hoạt cho người dùng.

Thư viện này cho phép import dữ liệu từ nhiều nguồn như CSV, cơ sở dữ liệu SQL, và các định dạng đặc biệt khác. Sau khi nhập dữ liệu, người dùng có thể thực hiện các thao tác như chọn lọc (subsetting), cắt nhỏ (slicing), lọc (filtering), hợp nhất (merging), nhóm (groupBy), sắp xếp (re-ordering), và tái cấu trúc (re-shaping) dữ liệu một cách dễ dàng. Pandas cũng hỗ trợ xử lý dữ liệu mất mát bằng cách loại bỏ hoặc thay thế bằng giá trị khác.

Ngoài ra, Pandas tích hợp tốt với các thư viện như Matplotlib, SciPy và Scikit-Learn, giúp mở rộng khả năng phân tích và trực quan hóa. Với hiệu suất cao, Pandas có thể xử lý khối lượng dữ liệu lớn hiệu quả, là công cụ không thể thiếu cho các nhà phân tích dữ liệu.

2.1.2 Cài đặt Pandas

Để cài đặt Pandas, bạn có thể sử dụng hai phương pháp chính: Anaconda và Pip

- Anaconda: là một nền tảng phổ biến cho khoa học dữ liệu và học máy, cung cấp một môi trường dễ sử dụng để quản lý các gói và môi trường ảo. Để cài đặt Pandas thông qua Anaconda, bạn có thể sử dụng lệnh sau: *conda install pandas*
- Pip: Pip là trình quản lý gói chuẩn cho Python. Nếu bạn không sử dụng Anaconda, bạn có thể cài đặt Pandas thông qua Pip bằng lệnh sau: *pip install pandas*

Sau khi cài đặt, để sử dụng Pandas chúng ta cần khai báo: `import pandas as pd`. Cách viết ngắn gọn này là tiêu chuẩn phổ biến, giúp mã nguồn dễ đọc và tuân theo các quy ước trong cộng đồng Python. Dù có thể đặt tên khác, nhưng dùng `pd` là lựa chọn tối ưu và được khuyến khích dễ đọc và tuân theo các quy ước chung trong tài liệu và ví dụ.

2.2 Numpy

2.2.1 Giới thiệu về Numpy

Numpy là thư viện cốt lõi của Python dành cho khoa học máy tính, hỗ trợ xử lý các mảng nhiều chiều với kích thước lớn. Thư viện này cung cấp các hàm tối ưu hóa để thao tác trên các mảng, giúp thực hiện các phép tính nhanh chóng và hiệu quả. NumPy đặc biệt hữu ích trong các bài toán liên quan đến đại số tuyến tính, xử lý tín hiệu, và tính toán ma trận, làm cho việc thao tác và phân tích dữ liệu số trở nên đơn giản và hiệu quả hơn.

2.2.2 Cài đặt Numpy

Để cài đặt NumPy, bạn có thể sử dụng hai phương pháp chính: Anaconda và Pip

- Anaconda: là một nền tảng phổ biến cho khoa học dữ liệu và học máy, cung cấp một môi trường dễ sử dụng để quản lý các gói môi trường ảo, bạn có thể tải bằng lệnh sau: `conda install numpy`
- Pip: Pip là trình quản lý gói chuẩn cho Python, bạn có thể tải bằng lệnh sau: `pip install numpy`

Sau khi cài đặt, cần khai báo thư viện NumPy để sử dụng trong mã nguồn Python. Thông thường, NumPy được khai báo gọn lại bằng lệnh `import numpy as np`

2.3 Matplotlib

2.3.1 Giới thiệu về Matplotlib

Để hỗ trợ các suy luận thống kê, trực quan hóa dữ liệu là bước không thể thiếu, và Matplotlib là một công cụ phổ biến trong Python. Đây là thư viện vẽ đồ thị mạnh mẽ, đặc biệt hữu ích khi kết hợp với Python và NumPy. Trong

Matplotlib, module Pyplot thường được sử dụng nhất, cung cấp giao diện thân thiện, tương tự MATLAB nhưng dựa trên Python và hoàn toàn mã nguồn mở. Matplotlib cho phép tạo ra nhiều loại biểu đồ như biểu đồ đường, biểu đồ thanh, biểu đồ phân tán và hơn thế nữa, giúp phân tích dữ liệu hiệu quả và trực quan. Với sự linh hoạt và khả năng tùy chỉnh cao, người dùng dễ dàng tạo các hình ảnh trực quan đáp ứng nhu cầu cụ thể của mình.

2.3.2 Cài đặt Matplotlib

Để cài đặt Matplotlib, bạn có thể sử dụng hai phương pháp chính: Anaconda và Pip.

- Anaconda: là một nền tảng phổ biến cho khoa học dữ liệu và học máy cung cấp một môi trường dễ sử dụng để quản lý các gói và môi trường ảo lệnh cài đặt qua Anaconda: *canda install matplolib*.
- Pip: Pip là trình quản lý gói chuẩn cho Python. Lệnh cài đặt qua Pip: *pip install matplotlib*.

Sau khi cài đặt, bạn có thể bắt đầu sử dụng Matplotlib trong mã nguồn Python bằng cách khai báo: `import matplotlib.pyplot as plt`. Đây là cách viết tắt phổ biến, được cộng đồng Python sử dụng rộng rãi, giúp mã nguồn dễ đọc và tuân theo các quy ước chuẩn trong tài liệu và ví dụ.

2.4 Random Forset

Random Forest là một thuật toán học máy mạnh mẽ thuộc nhóm học máy giám sát (supervised learning) và thường được sử dụng cho các bài toán phân loại và hồi quy. Thuật toán này kết hợp nhiều cây quyết định (decision trees) để tạo thành một "rừng" cây, và kết quả cuối cùng được đưa ra dựa trên kết quả từ tất cả các cây trong rừng.

2.4.1 Cấu trúc của Random Forest

Random Forest là một bộ phân loại học theo phương thức ensemble learning, tức là nó xây dựng nhiều mô hình đơn lẻ (các cây quyết định) và kết hợp chúng lại để đưa ra dự đoán chính xác hơn.

- Cây quyết định (Decision Trees): Mỗi cây quyết định trong Random Forest là một cây riêng biệt được xây dựng bằng cách sử dụng một tập con ngẫu nhiên của dữ liệu và các đặc trưng.
- Quy trình Ensemble: Sau khi các cây quyết định được xây dựng, kết quả của mỗi cây sẽ được lấy (dựa trên đa số phiếu bầu cho phân loại hoặc giá trị trung bình cho hồi quy) để đưa ra dự đoán cuối cùng.

2.4.2 Cách thức hoạt động của Random Forest

a) Tạo ra các cây quyết định:

- Tạo ra một tập hợp các mẫu ngẫu nhiên từ tập huấn luyện bằng cách sử dụng Bootstrap sampling (hay còn gọi là sampling với thay thế).
- Tại mỗi nút trong cây, thay vì thử tất cả các đặc trưng để chia, thuật toán chỉ chọn một tập con ngẫu nhiên của các đặc trưng để tạo sự đa dạng giữa các cây.

b) Dự đoán từ mỗi cây:

- Phân loại: mỗi cây đưa ra một dự đoán phân loại. Random Forest sẽ chọn kết quả dự đoán mà cây quyết định chiếm số phiếu bầu cao nhất.
- Hồi quy: kết quả dự đoán là giá trị trung bình của tất cả các cây trong rừng

2.4.3 Ứng dụng của Random Forest

- Phân loại: ví dụ như phân loại email thành thư rác và không phải thư rác, phân loại khách hàng thành nhóm có thể mua và không thể mua, hoặc phân loại nhân viên có khả năng nghỉ việc hay không.
- Hồi quy: ví dụ như dự đoán giá trị của bất động sản, giá cổ phiếu, hay giá trị của các biến liên tục khác.
- Phân tích tầm quan trọng đặc trưng: Random Forest có thể được sử dụng để phân tích các yếu tố quan trọng trong tập dữ liệu, giúp xác định các đặc trưng có ảnh hưởng mạnh đến dự đoán.

2.5 Logistic Regression

Logistic Regression là một thuật toán học máy cơ bản và phổ biến thuộc nhóm học máy giám sát (supervised learning). Mặc dù có tên là "hồi quy", nhưng thuật toán này chủ yếu được sử dụng cho các bài toán phân loại nhị phân (binary classification), nơi kết quả dự đoán chỉ nằm trong hai trạng thái (ví dụ: Nghỉ việc hoặc Ở lại).

2.5.1 Cấu trúc của Logistic Regression

Logistic Regression hoạt động dựa trên việc thiết lập mối quan hệ giữa các biến độc lập (các chỉ số nhân sự) và xác suất xảy ra của một biến phụ thuộc (khả năng nhân viên nghỉ việc).

- Hàm Tuyến tính: Thuật toán bắt đầu bằng cách tính tổng trọng số của các đặc trưng đầu vào (như tuổi, thu nhập hàng tháng, số năm làm việc, làm thêm giờ...).
- Hàm Sigmoid: Điểm khác biệt cốt lõi là việc đưa kết quả của hàm tuyến tính qua hàm Sigmoid (S-shape). Hàm này có nhiệm vụ nén mọi giá trị đầu vào vào khoảng từ 0 đến 1, đại diện cho mức xác suất nhân viên nghỉ việc.

2.5.2 Cách thức hoạt động của Logistic Regression

a) Tính toán xác suất:

- Thuật toán gán cho mỗi đặc trưng đầu vào một trọng số (w) thể hiện mức độ ảnh hưởng của nó. Ví dụ, yếu tố làm thêm giờ (OverTime) thường có trọng số cao vì nó ảnh hưởng mạnh đến quyết định nghỉ việc của nhân viên.
- Hàm Sigmoid sẽ tính toán dựa trên các trọng số này để trả về một con số xác suất (ví dụ: 0.75 tương đương 75% khả năng nhân viên nghỉ việc).

b) Quyết định phân loại:

- Một ngưỡng (thông thường là 0.5) được thiết lập để đưa ra kết luận cuối cùng.
- Nếu xác suất ≥ 0.5 mô hình dự đoán là “Có khả năng nghỉ việc”.
- Nếu xác suất < 0.5 mô hình dự đoán là “Ở lại công ty”.

2.5.3 Ứng dụng của Logistic Regression

- Phân tích nhân sự: Dự đoán khả năng nhân viên nghỉ việc dựa trên các yếu tố như thu nhập, thâm niên, mức độ hài lòng và điều kiện làm việc.
- Quản trị doanh nghiệp: Hỗ trợ bộ phận nhân sự xây dựng chiến lược giữ chân nhân viên và giảm tỷ lệ nghỉ việc.
- Phân tích tác động: Giúp xác định các yếu tố rủi ro hàng đầu (như làm thêm giờ, mức độ hài lòng công việc hoặc thu nhập) ảnh hưởng như thế nào đến khả năng nhân viên nghỉ việc thông qua các hệ số hồi quy.

2.5.4 Loss Function

Hàm mất mát trong hồi quy tuyến tính là Mean Squared Error (MSE), tính toán trung bình bình phương sai số giữa các giá trị thực tế và giá trị dự đoán:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

2.5.5 Các loại Logistic Regression

- Single Logistic Regression (hồi quy Logistic đơn biến): Khi có chỉ một biến độc lập (feature), mô hình trở thành một đường thẳng trong không gian 2 chiều.
- Multiple Logistic Regression (hồi quy Logistic đa biến): Khi có nhiều biến độc lập, mô hình trở thành một siêu phẳng trong không gian n chiều.

CHƯƠNG 3: NỘI DUNG THỰC HIỆN VÀ KẾT QUẢ

3.1 Mã nguồn tiền xử lý dữ liệu

```
print(df.info())
```

```
print(df.describe())
```

```
print(df.dtypes)
```

```
print(df.nunique())
```

Đầu tiên em sử dụng hàm `df.info()` để kiểm tra các feature nào bị thiếu và có kiểu dữ liệu thế nào. Em hiển thị lại hàm `df` và dùng lại hàm `info` để kiểm tra xem dữ liệu. Và hình ảnh dưới là kết quả sau khi đã điền các dữ liệu:

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 35 columns):
 #   Column                Non-Null Count  Dtype  
---  --
 0   Age                   1470 non-null  int64  
 1   Attrition             1470 non-null  object  
 2   BusinessTravel        1470 non-null  object  
 3   DailyRate             1470 non-null  int64  
 4   Department            1470 non-null  object  
 5   DistanceFromHome      1470 non-null  int64  
 6   Education              1470 non-null  int64  
 7   EducationField        1470 non-null  object  
 8   EmployeeCount          1470 non-null  int64  
 9   EmployeeNumber        1470 non-null  int64  
10   EnvironmentSatisfaction 1470 non-null  int64  
11   Gender                 1470 non-null  object  
12   HourlyRate            1470 non-null  int64  
13   JobInvolvement         1470 non-null  int64  
14   JobLevel              1470 non-null  int64  
15   JobRole               1470 non-null  object  
16   JobSatisfaction        1470 non-null  int64  
17   MaritalStatus          1470 non-null  object  
18   MonthlyIncome          1470 non-null  int64  
19   MonthlyRate           1470 non-null  int64  
20   NumCompaniesWorked     1470 non-null  int64  
21   Over18                1470 non-null  object  
22   OverTime              1470 non-null  object  
23   PercentSalaryHike      1470 non-null  int64  
24   PerformanceRating      1470 non-null  int64  
25   RelationshipSatisfaction 1470 non-null  int64  
26   StandardHours          1470 non-null  int64  
27   StockOptionLevel       1470 non-null  int64  
28   TotalWorkingYears      1470 non-null  int64
```

Hình 3.1: Thông tin từng cột của Dataset

a) Xóa các cột: `EmployeeNumber`, `EmployeeCount`, `StandardHours`, `Over18`.

```
df.drop(['EmployeeNumber', 'EmployeeCount', 'StandardHours', 'Over18'], axis=1, inplace=True)
```

(1470, 31)

Hình 3.2: Dữ liệu sau khi xóa cột dư thừa

- Đoạn mã dùng để xóa bỏ những cột dữ liệu dư thừa không có tác dụng trong quá trình dự đoán mô hình.

b) Kiểm tra dữ liệu trùng lặp

```
print("\nDuplicate values")
print(df.duplicated().sum())
```

```
Duplicate values
0
```

Hình 3.3: Kiểm tra giá trị trùng lặp

- Đếm số lần xuất hiện và cộng tổng số lượng các giá trị trùng lặp bằng hàm `duplicated().sum()`

c) Kiểm tra giá trị thiếu

```
print("\nMissing values:")
print(df.isnull().sum())
```

✓ 0.0s

Missing values	
Age	0
Attrition	0
BusinessTravel	0
DailyRate	0
Department	0
DistanceFromHome	0
Education	0
EducationField	0
EnvironmentSatisfaction	0
Gender	0
HourlyRate	0
JobInvolvement	0
JobLevel	0
JobRole	0
JobSatisfaction	0
MaritalStatus	0
MonthlyIncome	0
MonthlyRate	0
NumCompaniesWorked	0
Overtime	0
PercentSalaryHike	0
PerformanceRating	0
RelationshipSatisfaction	0
StockOptionLevel	0
TotalWorkingYears	0
TrainingTimesLastYear	0
WorkLifeBalance	0
YearsAtCompany	0
YearsInCurrentRole	0
YearsSinceLastPromotion	0
YearsWithCurrManager	0
dtype: int64	

Hình 3.4: Kiểm tra giá trị thiếu

- Đếm số lần xuất hiện và cộng tổng số lượng các giá trị thiếu bằng hàm `isnull().sum()`

3.2 Mã nguồn thống kê dữ liệu

- a) Thống kê số lượng nhân viên nam và nữ


```
df['Gender'].value_counts()
```

```
Gender
Male      882
Female    588
Name: count, dtype: int64
```

Hình 3.5: Mã nguồn đếm số lượng giới tính

- Đếm số lượng nhân viên nam và nữ trong cột 'Gender': sử dụng phương thức `value_counts()` để đếm số lần xuất hiện từng giá trị trong cột 'Gender'

b) Thống kê số lượng nhân viên nghỉ việc trong dữ liệu

```
df['Attrition'].value_counts()
```

```
Attrition
No      1233
Yes      237
Name: count, dtype: int64
```

Hình 3.6: Mã nguồn đếm số lượng nhân viên nghỉ việc

- Đếm số lượng nhân viên nghỉ việc và ở lại trong cột 'Attrition': sử dụng phương thức `value_counts()` để đếm số lần xuất hiện từng giá trị trong cột 'Attrition'

c) Thống kê số lượng tình trạng hôn nhân của nhân viên trong dữ liệu

```
df['MaritalStatus'].value_counts()
```

```
MaritalStatus
Married      673
Single      470
Divorced     327
Name: count, dtype: int64
```

Hình 3.7: Mã nguồn đếm số lượng tình trạng hôn nhân của nhân viên

- Đếm số lượng tình trạng hôn nhân của nhân viên trong cột 'MaritalStatus': sử dụng phương thức `value_counts()` để đếm số lần xuất hiện từng giá trị trong cột 'MaritalStatus'

d) Thống kê nhân viên có mức lương hàng tháng cao hơn mức trung bình

```
mean_income = df['MonthlyIncome'].mean()
high_income_employees = df[df['MonthlyIncome'] > mean_income]

count_total = len(df)
count_high = len(high_income_employees)
percentage = (count_high / count_total) * 100

print(f"=== THỐNG KÊ THU NHẬP ===")
print(f"- Thu nhập trung bình toàn công ty: {mean_income:,.2f}")
print(f"- Số nhân viên có thu nhập > trung bình: {count_high}")
print(f"- Tỷ lệ phần trăm: {percentage:,.2f}%")

=== THỐNG KÊ THU NHẬP ===
- Thu nhập trung bình toàn công ty: 6,502.93
- Số nhân viên có thu nhập > trung bình: 493
- Tỷ lệ phần trăm: 33.54%
```

Hình 3.8: Mã nguồn đếm số lượng nhân viên có lương hàng tháng cao hơn mức trung bình

- Đếm số lượng nhân viên có mức lương hàng tháng 'MonthlyIncome' cao hơn mức trung bình 6,502.93 so sánh với mean_income để biết tỉ lệ nhân viên có mức lương hàng tháng cao hơn mức trung bình.

e) Thống kê số lượng vị trí công việc của nhân viên trong dữ liệu

```
df['JobRole'].value_counts()

JobRole
Sales Executive      326
Research Scientist   292
Laboratory Technician 259
Manufacturing Director 145
Healthcare Representative 131
Manager              102
Sales Representative   83
Research Director      80
Human Resources        52
Name: count, dtype: int64
```

Hình 3.9: Mã nguồn đếm số lượng vị trí công việc của nhân viên

- Đếm số lượng vị trí công việc của nhân viên trong cột 'JobRole': sử dụng phương thức `value_counts()` để đếm số lần xuất hiện từng giá trị trong cột 'JobRole'

f) Thống kê số lượng cấp bậc công việc của nhân viên trong dữ liệu

```
df['JobLevel'].value_counts()
```

```
JobLevel
1      543
2      534
3      218
4      106
5       69
Name: count, dtype: int64
```

Hình 3.10: Mã nguồn đếm số lượng cấp bậc công việc của nhân viên

- Đếm số lượng cấp bậc công việc của nhân viên trong cột 'JobLevel': sử dụng phương thức `value_counts()` để đếm số lần xuất hiện từng giá trị trong cột 'JobLevel'

g) Thống kê nhân viên có khoảng cách từ nhà đến công ty xa hơn mức trung bình

```
mean_distance = df['DistanceFromHome'].mean()

far_employees = df[df['DistanceFromHome'] > mean_distance]
count_far = len(far_employees)

total_employees = len(df)
percentage = (count_far / total_employees) * 100

print(f"- Khoảng cách trung bình: {mean_distance:.2f}km")
print(f"- Số nhân viên đi làm xa hơn mức trung bình: {count_far}")
print(f"- Tỷ lệ phần trăm: {percentage:.2f}%")
```

- Khoảng cách trung bình: 9.19km
- Số nhân viên đi làm xa hơn mức trung bình: 530
- Tỷ lệ phần trăm: 36.05%

Hình 3.11: Mã nguồn đếm số lượng nhân viên có khoảng cách từ nhà đến công ty xa hơn mức trung bình

- Đếm số lượng nhân viên đi làm xa công ty 'DistanceFromHome' cao hơn mức trung bình 9,19km so sánh với mean_distance để biết tỉ lệ nhân viên đi làm xa công ty hơn mức trung bình.

3.3 Mã nguồn trực quan hóa

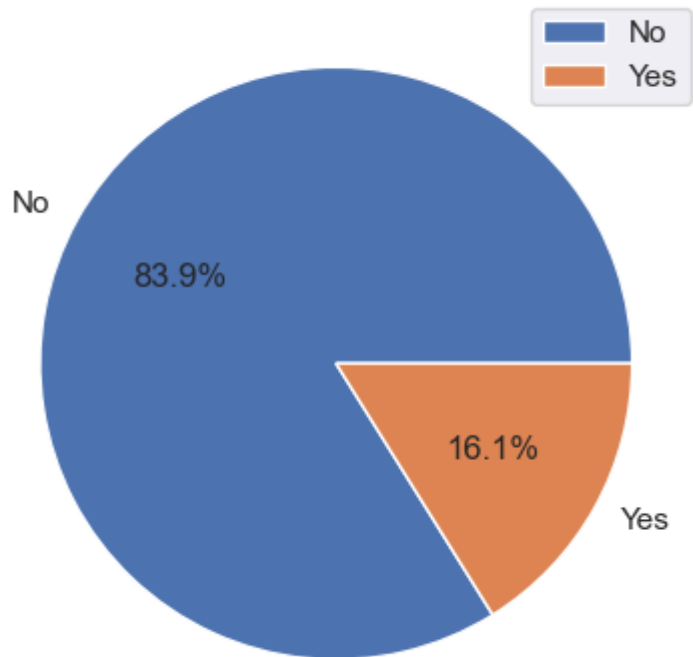
1. Biểu đồ phân phối tỉ lệ nhân viên nghỉ việc

```
plt.pie(df['Attrition'].value_counts(), labels=df['Attrition'].value_counts().index, autopct='%1.1f%%')
plt.title('Phân phối của Attrition ((No)Stayed, (Yes):Lefted)')
plt.legend()
plt.show()
```

Hình 3.12: Mã nguồn biểu đồ phân phối tỉ lệ nhân viên nghỉ việc

- Đếm số lượng nhân viên nghỉ việc và ở lại trong dữ liệu cột 'Attrition' bằng hàm value_counts().
- plt.pie thiết lập tên các vùng dữ liệu (labels) và hiển thị tỉ lệ phần trăm lấy một số thập phân.
- plt.title() đặt tên tiêu đề cho biểu đồ.
- plt.legend() hiển thị bảng chú thích các thành phần.
- plt.show() hiển thị biểu đồ lên màn hình.

Phân phối của Attrition ((No)Stayed, (Yes):Lefted)



Hình 3.13: Biểu đồ phân phối tỉ lệ nhân viên nghỉ việc

2. Biểu đồ tương quan

```
df_viz = df.copy()
if df_viz['Attrition'].dtype == 'object':
    df_viz['Attrition'] = df_viz['Attrition'].apply(lambda x: 1 if x == 'Yes' else 0)
corr_matrix = df_viz.corr(numeric_only=True)
k = 24
corr_matrix = df_viz.corr(numeric_only=True)

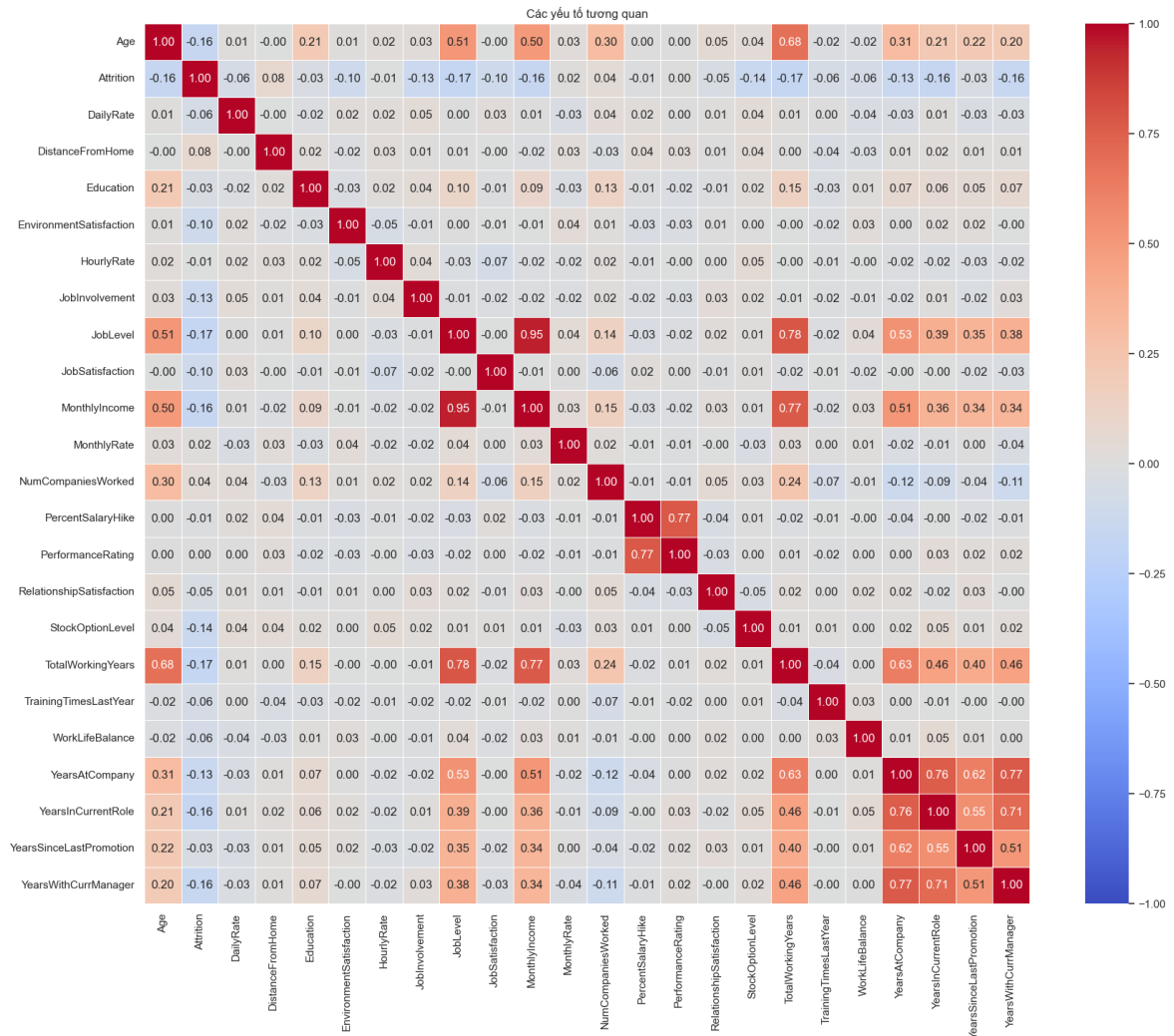
plt.figure(figsize=(20, 16))
sns.set(font_scale=1.0)

sns.heatmap(corr_matrix,
            annot=True,
            fmt=".2f",
            cmap='coolwarm',
            linewidths=0.5,
            vmin=-1, vmax=1)
plt.title('Các yếu tố tương quan')
plt.show()
```

Hình 3.14: Mã nguồn biểu đồ heatmap

- Tạo bản sao dữ liệu sang biến `df_viz` bằng `df.copy()` để xử lý mà không làm ảnh hưởng đến dữ liệu gốc

- Sử dụng hàm `apply(lambda...)` để chuyển đổi cột mục tiêu 'Attrition' từ dạng chữ (Yes/No) sang dạng số (1/0) nhằm phục vụ việc tính toán toán học
- Tính toán hệ số tương quan giữa tất cả các cột dữ liệu số bằng hàm `df_viz.corr(numeric_only=True)`
- `plt.figure(figsize=(20, 16))` thiết lập kích thước khung hình lớn để hiển thị rõ các ô dữ liệu
- Sử dụng `sns.heatmap` để vẽ biểu đồ nhiệt với các tham số quan trọng:
- `annot=True`: Hiển thị giá trị hệ số tương quan cụ thể lên từng ô
- `fmt=".2f"`: Định dạng con số hiển thị với 2 chữ số thập phân
- `cmap='coolwarm'`: Sử dụng bảng màu Xanh-Đỏ (Lạnh-Nóng) để dễ dàng phân biệt tương quan âm và dương
- `vmin=-1, vmax=1`: Cố định thang đo giá trị từ -1 đến 1 (giới hạn của hệ số tương quan Pearson)
- `plt.show()` hiển thị biểu đồ lên màn hình để phân tích các yếu tố ảnh hưởng mạnh đến biến Attrition



Hình 3.15: Biểu đồ tương quan

3. Biểu đồ phân phối tuổi theo trạng thái nghỉ việc

```
plt.figure(figsize=(10, 6))

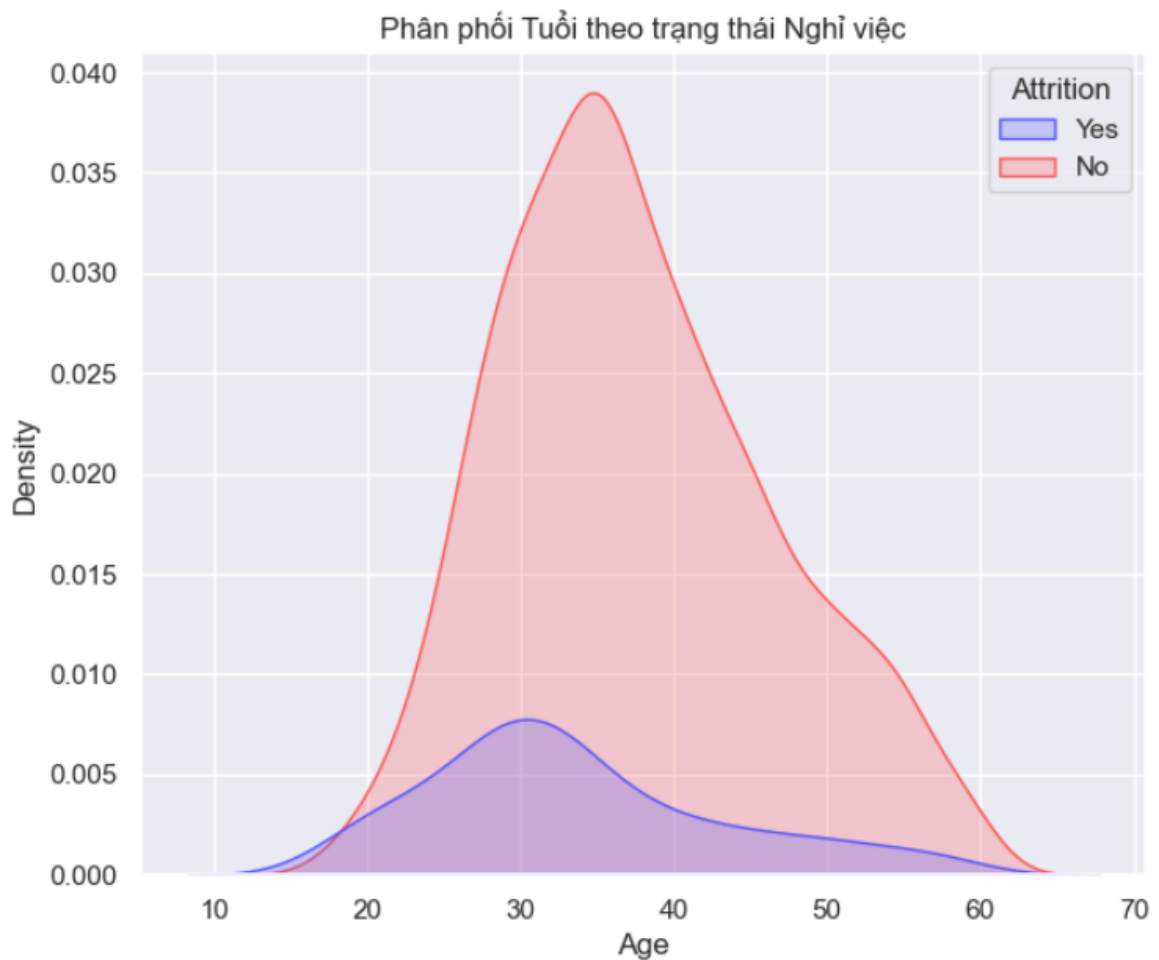
sns.kdeplot(data=df, x='Age', hue='Attrition', fill=True, palette='seismic')
plt.title('Phân phối Tuổi theo trạng thái Nghỉ việc')

plt.show()
```

Hình 3.16: Biểu đồ phân phối tuổi theo trạng thái nghỉ việc

- `plt.figure(figsize=(10, 6))` thiết lập kích thước khung hình hiển thị biểu đồ
- Sử dụng hàm `sns.kdeplot` để vẽ biểu đồ mật độ xác suất (Kernel Density Estimate) giúp so sánh phân phối giữa các nhóm

- Tham số `x='Age'` xác định dữ liệu trục hoành là Tuổi và `hue='Attrition'` để phân tách đường cong theo hai nhóm nhân viên (Nghỉ việc/Ở lại)
- `fill=True` để tô màu vùng dưới đường cong và `palette='seismic'` sử dụng bảng màu tương phản giúp dễ dàng quan sát vùng giao nhau
- `plt.title()` đặt tên tiêu đề mô tả nội dung biểu đồ
- `plt.show()` hiển thị biểu đồ hoàn chỉnh lên màn hình



Hình 3.17: Biểu đồ phân phối tuổi theo trạng thái nghỉ việc

4. Biểu đồ phân phối thu nhập theo trạng thái nghỉ việc

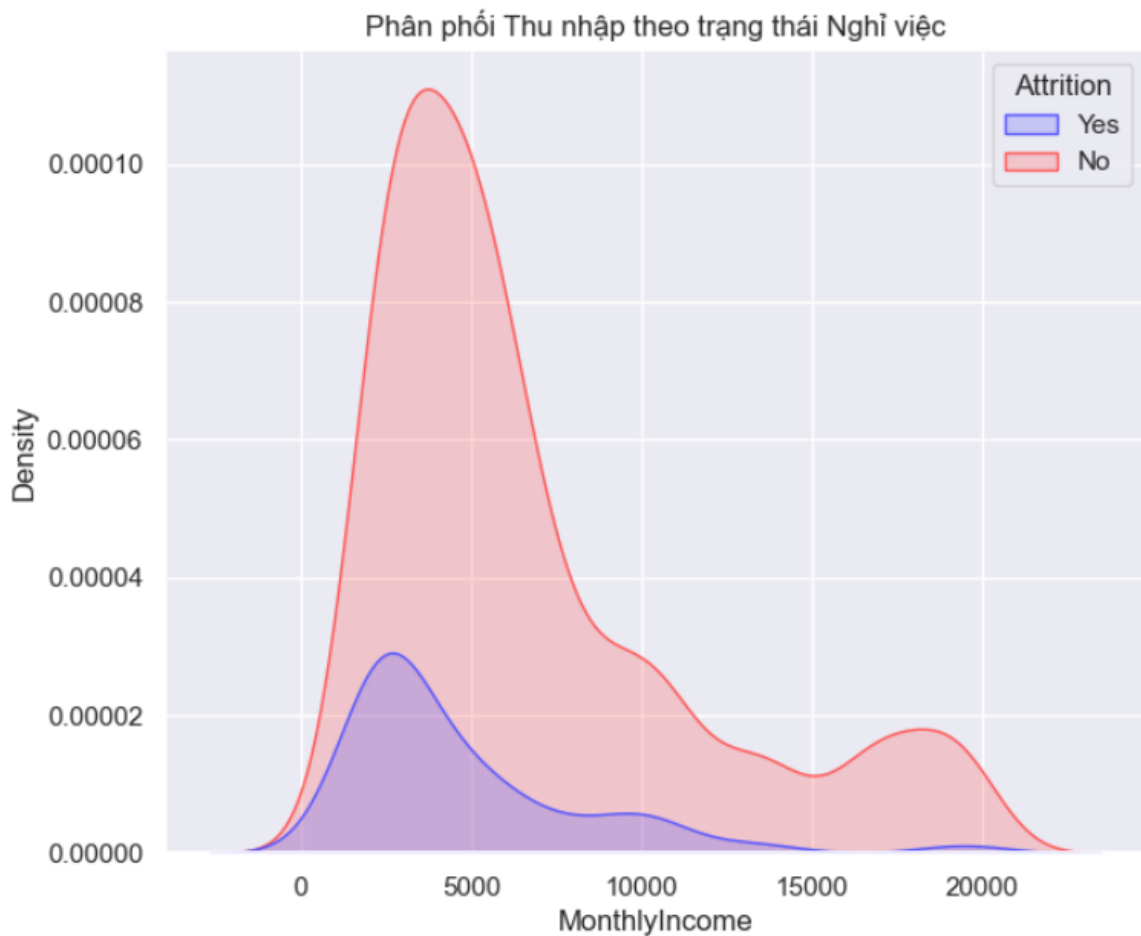

```
plt.figure(figsize=(10, 6))

sns.kdeplot(data=df, x='MonthlyIncome', hue='Attrition', fill=True, palette='seismic')
plt.title('Phân phối Thu nhập theo trạng thái Nghỉ việc')

plt.show()
```

Hình 3.18: Mã nguồn biểu đồ phân phối thu nhập theo trạng thái nghỉ việc

- `plt.figure(figsize=(10, 6))` thiết lập kích thước khung hình hiển thị biểu đồ
- Sử dụng hàm `sns.kdeplot` để vẽ biểu đồ mật độ xác suất (Kernel Density Estimate) giúp so sánh phân phối giữa các nhóm
- Tham số `x='MonthlyIncome'` xác định dữ liệu trục hoành là Thu nhập và `hue='Attrition'` để phân tách đường cong theo hai nhóm nhân viên
- `fill=True` để tô màu vùng dưới đường cong và `palette='seismic'` sử dụng bảng màu tương phản giúp dễ dàng quan sát vùng giao nhau
- `plt.title()` đặt tên tiêu đề mô tả nội dung biểu đồ
- `plt.show()` hiển thị biểu đồ hoàn chỉnh lên màn hình



Hình 3.19: Biểu đồ phân phối thu nhập theo trạng thái nghỉ việc

5. Biểu đồ phân phối khoảng cách từ nhà đến công ty theo trạng thái nghỉ việc

```
plt.figure(figsize=(10, 6))

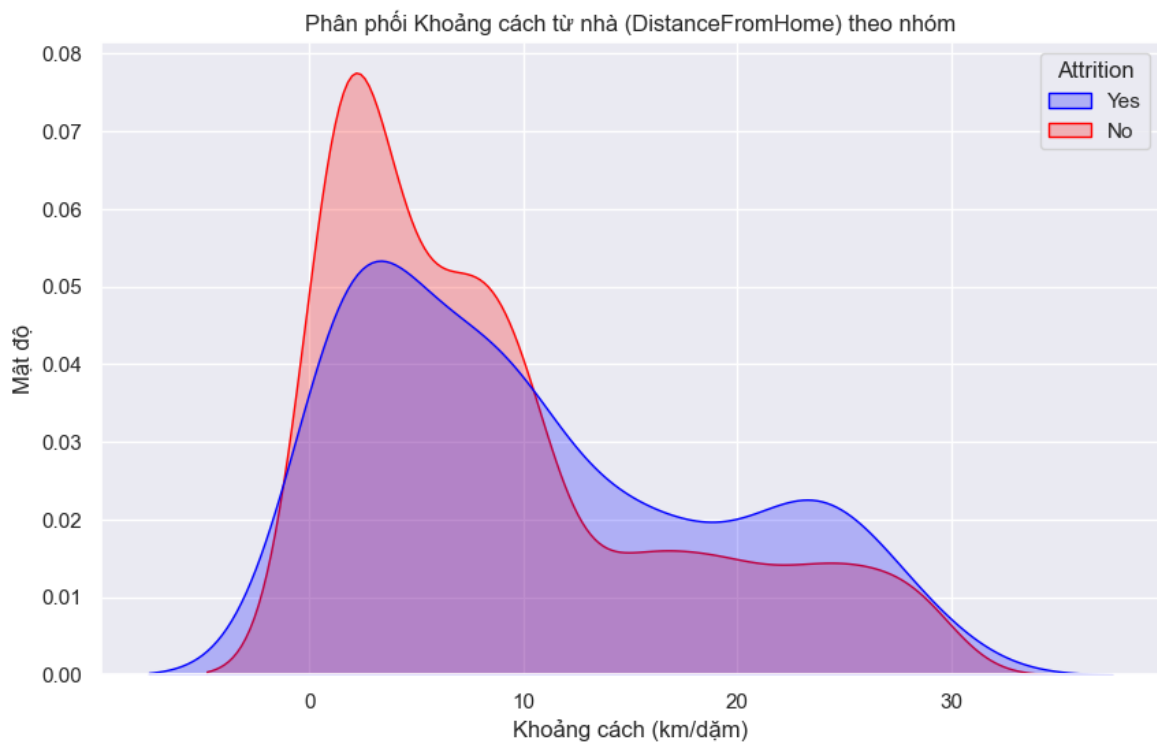
sns.kdeplot(data=df, x='DistanceFromHome', hue='Attrition', fill=True, palette=['blue', 'red'], common_norm=False)

plt.title('Phân phối Khoảng cách từ nhà (DistanceFromHome) theo nhóm')
plt.xlabel('Khoảng cách (km/dặm)')
plt.ylabel('Mật độ')
plt.show()
```

Hình 3.20: Mã nguồn biểu đồ phân phối khoảng cách từ nhà đến công ty theo trạng thái nghỉ việc

- `plt.figure(figsize=(10, 6))` thiết lập kích thước khung hình hiển thị biểu đồ
- Sử dụng hàm `sns.kdeplot` để vẽ biểu đồ mật độ xác suất (KDE) thể hiện phân phối dữ liệu

- Tham số `x='DistanceFromHome'` xác định dữ liệu trục hoành là Khoảng cách từ nhà và `hue='Attrition'` để phân tách đường cong theo hai nhóm nhân viên
- `fill=True` để tô màu vùng dưới đường cong và `palette=['blue', 'red']` tùy chỉnh màu sắc riêng biệt (Xanh/Đỏ) cho hai nhóm
- `common_norm=False` là tham số quan trọng giúp chuẩn hóa mật độ riêng cho từng nhóm, giúp so sánh hình dạng phân phối chính xác ngay cả khi số lượng mẫu chênh lệch lớn (do dữ liệu mất cân bằng)
- `plt.title()`, `plt.xlabel()`, `plt.ylabel()` lần lượt đặt tên tiêu đề, nhãn trục hoành và trục tung cho biểu đồ giúp người xem dễ hiểu
- `plt.show()` hiển thị biểu đồ hoàn chỉnh lên màn hình



Hình 3.21: Biểu đồ phân phối khoảng cách từ nhà đến công ty theo trạng thái nghỉ việc

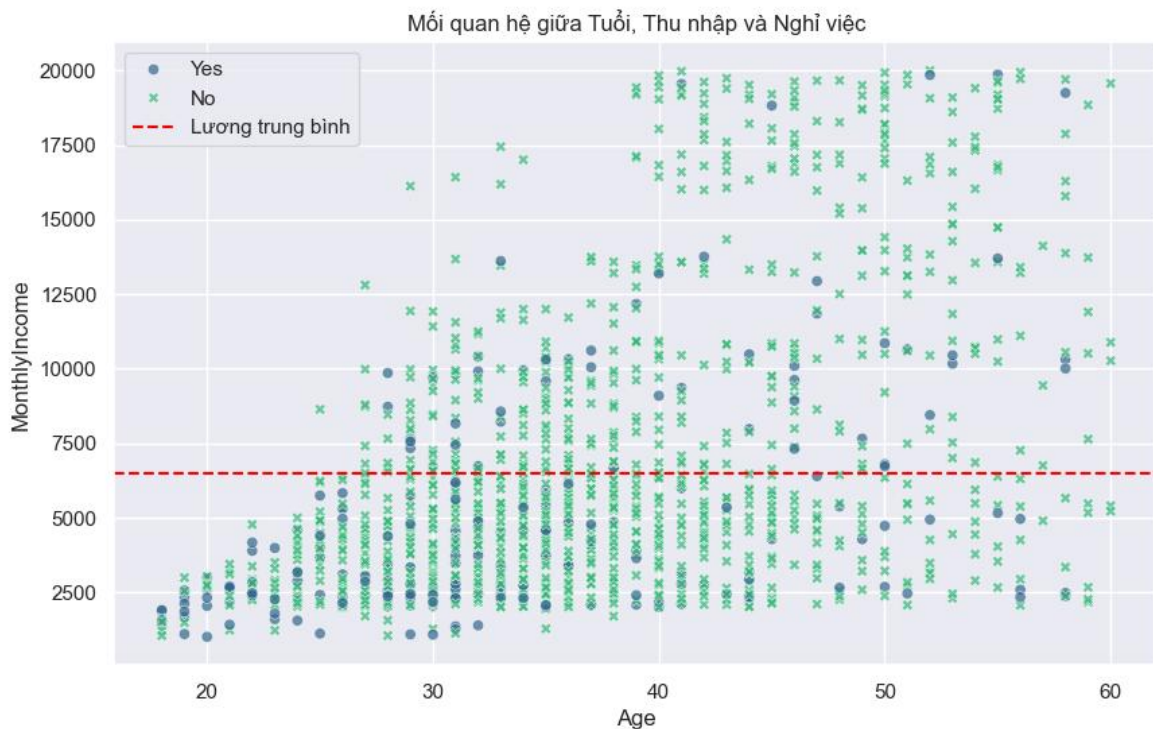
6. Biểu đồ scatterplot mối quan hệ giữa tuổi, thu nhập và nghỉ việc

```
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x='Age', y='MonthlyIncome', hue='Attrition', style='Attrition', alpha=0.7, palette='viridis')
plt.title('Mối quan hệ giữa Tuổi, Thu nhập và Nghỉ việc')

plt.axhline(y=df['MonthlyIncome'].mean(), color='red', linestyle='--', label='Lương trung bình')
plt.legend()
plt.show()
```

Hình 3.22: Mã nguồn biểu đồ scatterplot mối quan hệ giữa tuổi, thu nhập và nghỉ việc

- `plt.figure(figsize=(10, 6))` thiết lập kích thước khung hình hiển thị biểu đồ
- Sử dụng hàm `sns.scatterplot` để vẽ biểu đồ phân tán giúp quan sát mối tương quan giữa 3 yếu tố: Tuổi, Thu nhập và Trạng thái nghỉ việc
- Tham số `x='Age'` và `y='MonthlyIncome'` xác định hai trục dữ liệu số
- `hue='Attrition'` và `style='Attrition'` vừa tô màu vừa thay đổi hình dạng điểm (ví dụ: tròn/chéo) để phân biệt rõ ràng nhóm Nghỉ việc và Ở lại
- `alpha=0.7` chỉnh độ trong suốt để dễ quan sát các điểm dữ liệu bị chồng lấn lên nhau
- `plt.title()` đặt tên tiêu đề mô tả nội dung biểu đồ
- Sử dụng hàm `plt.axhline` để vẽ một đường kẻ ngang tham chiếu (Reference line)
- `y=df['MonthlyIncome'].mean()` xác định vị trí đường kẻ tại mức lương trung bình của toàn công ty
- `color='red'` và `linestyle='--'` thiết lập đường kẻ màu đỏ và nét đứt để làm nổi bật so với các điểm dữ liệu
- `plt.legend()` hiển thị bảng chú thích và `plt.show()` xuất biểu đồ ra màn hình



Hình 3.23: Biểu đồ phân tán mối quan hệ giữa tuổi, thu nhập và nghỉ việc

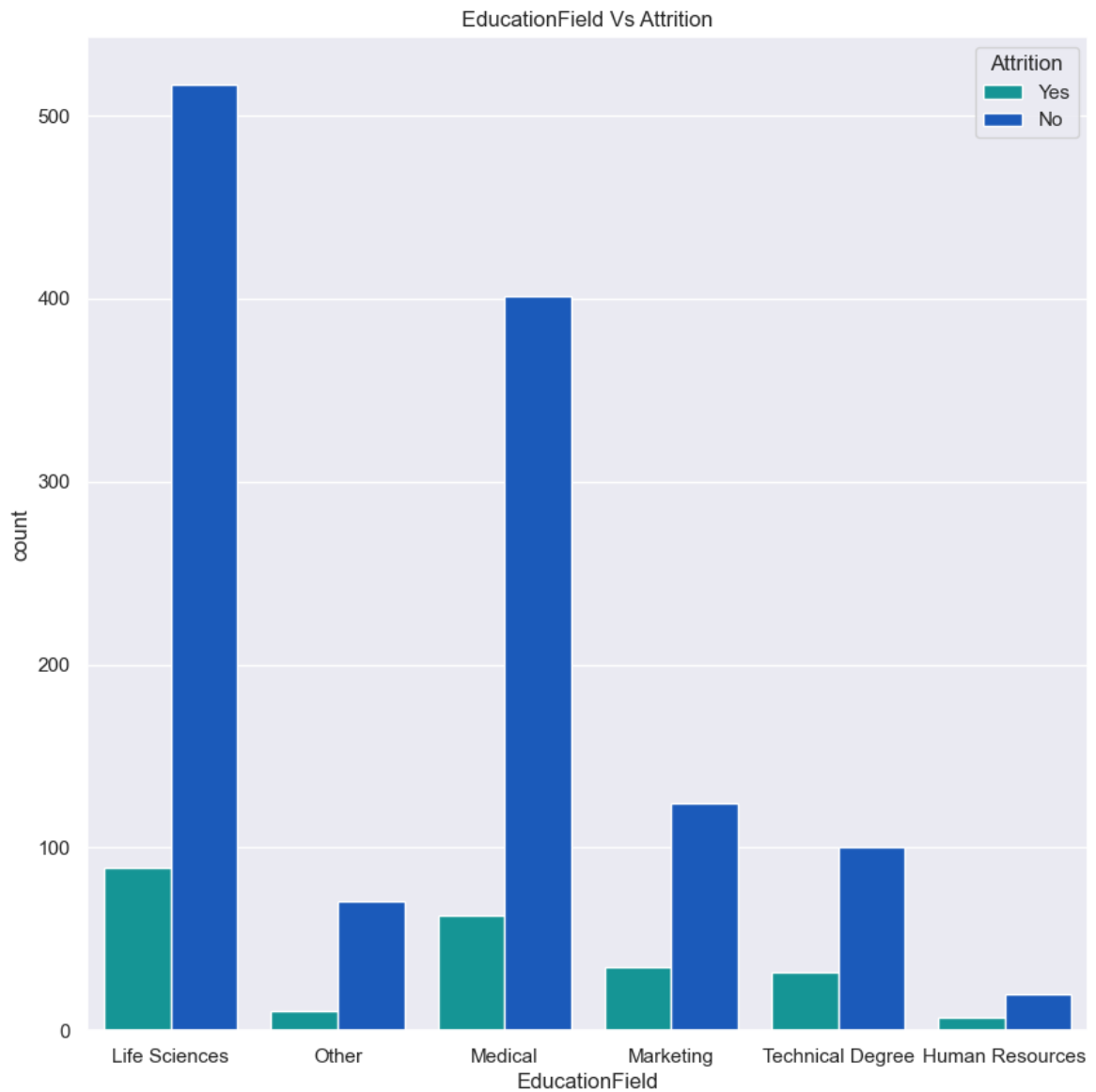
7. Biểu đồ phân bố lĩnh vực giáo dục theo trạng thái nghỉ việc

```
plt.figure(figsize=(10,10))
sns.countplot(x= 'EducationField', data =df ,palette='winter_r',hue='Attrition')
plt.title('EducationField Vs Attrition')
plt.show()
```

Hình 3.24: Mã nguồn biểu đồ phân bố lĩnh vực giáo dục theo trạng thái nghỉ việc

- `plt.figure(figsize=(10,10))` thiết lập kích thước khung hình vuông (10x10 inch) tạo không gian rộng rãi để hiển thị đầy đủ tên các chuyên ngành mà không bị chồng chéo
- Sử dụng hàm `sns.countplot` để thống kê và so sánh số lượng nhân viên giữa các lĩnh vực đào tạo khác nhau
- Tham số `x='EducationField'` đặt các chuyên ngành (như Y tế, Kỹ thuật, Marketing...) lên trục hoành
- `hue='Attrition'` là tham số quan trọng nhất giúp chia tách mỗi chuyên ngành thành 2 cột (Nghỉ việc/Ở lại), giúp nhận diện xem ngành nào có tỷ lệ nhân viên rời đi cao nhất

- `palette='winter_r'` sử dụng bảng màu tông lạnh (Xanh lam - Xanh lục) chuyển màu ngược để tạo giao diện trực quan
- `plt.title()` đặt tên tiêu đề và `plt.show()` hiển thị biểu đồ hoàn chỉnh



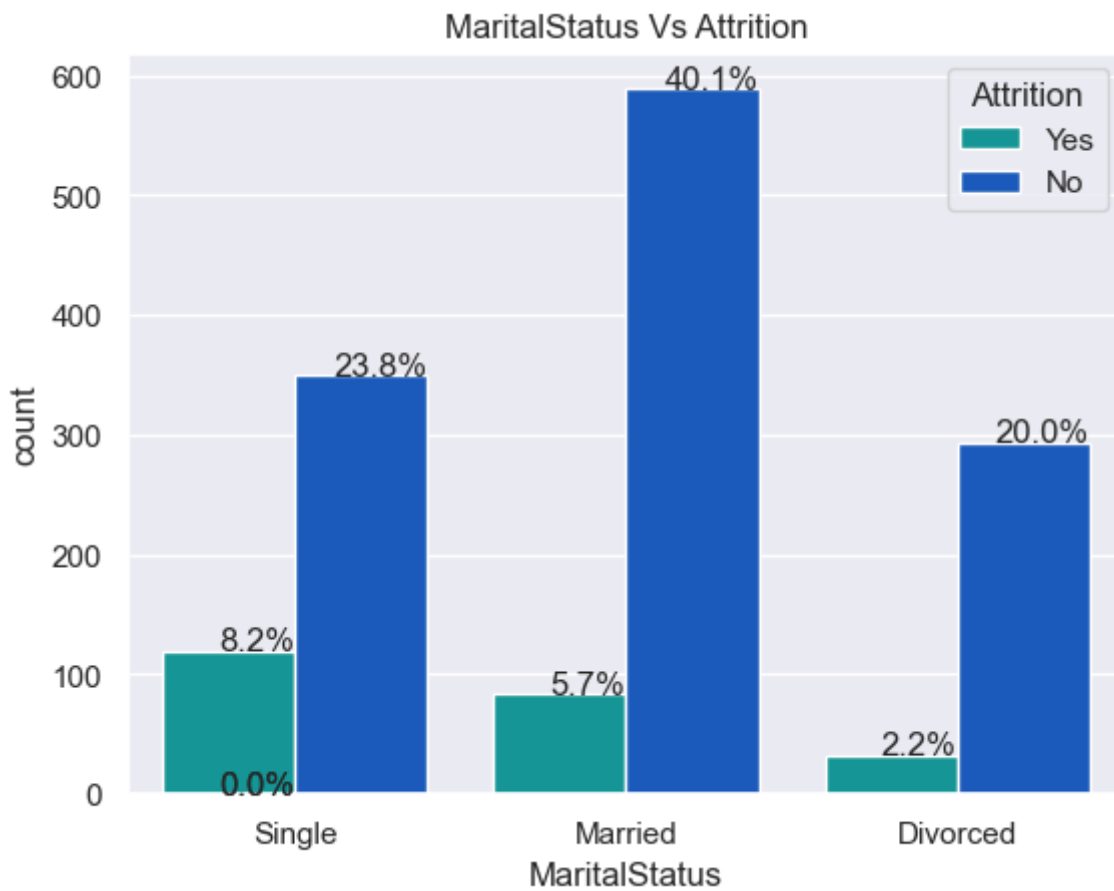
Hình 3.25: Biểu đồ phân bố lĩnh vực giáo dục theo trạng thái nghỉ việc

8. Biểu đồ phân bố tình trạng hôn nhân theo trạng thái nghỉ việc

```
a=sns.countplot(x= 'MaritalStatus' ,data =df ,palette='winter_r',hue='Attrition')
total = float(len(df))
plt.title('MaritalStatus Vs Attrition')
for p in a.patches:
    percentage = '{:.1f}%'.format(100 * p.get_height()/total)
    x = p.get_x() + p.get_width()
    y = p.get_height()
    a.annotate(percentage, (x, y),ha='right')
plt.show()
```

Hình 3.26: Mã nguồn biểu đồ phân bố tình trạng hôn nhân theo trạng thái nghỉ việc

- Tính toán biến total bằng tổng số dòng dữ liệu len(df) để làm mẫu số chung cho việc tính tỷ lệ phần trăm
- Sử dụng vòng lặp for để duyệt qua từng cột (patch) trong biểu đồ, tính toán giá trị phần trăm dựa trên chiều cao của cột đó
- sns.countplot vẽ biểu đồ cột so sánh số lượng nhân viên theo Tình trạng hôn nhân (MaritalStatus), phân tách màu sắc bởi biến Attrition (Nghỉ/Ở)
- a.annotate là lệnh quan trọng giúp viết trực tiếp con số phần trăm (percentage) lên đỉnh của từng cột tại toạ độ (x, y), giúp người xem nắm bắt số liệu chính xác
- plt.title() đặt tên tiêu đề và plt.show() hiển thị biểu đồ hoàn chỉnh



Hình 3.27: Biểu biểu đồ phân bố tình trạng hôn nhân theo trạng thái nghỉ việc

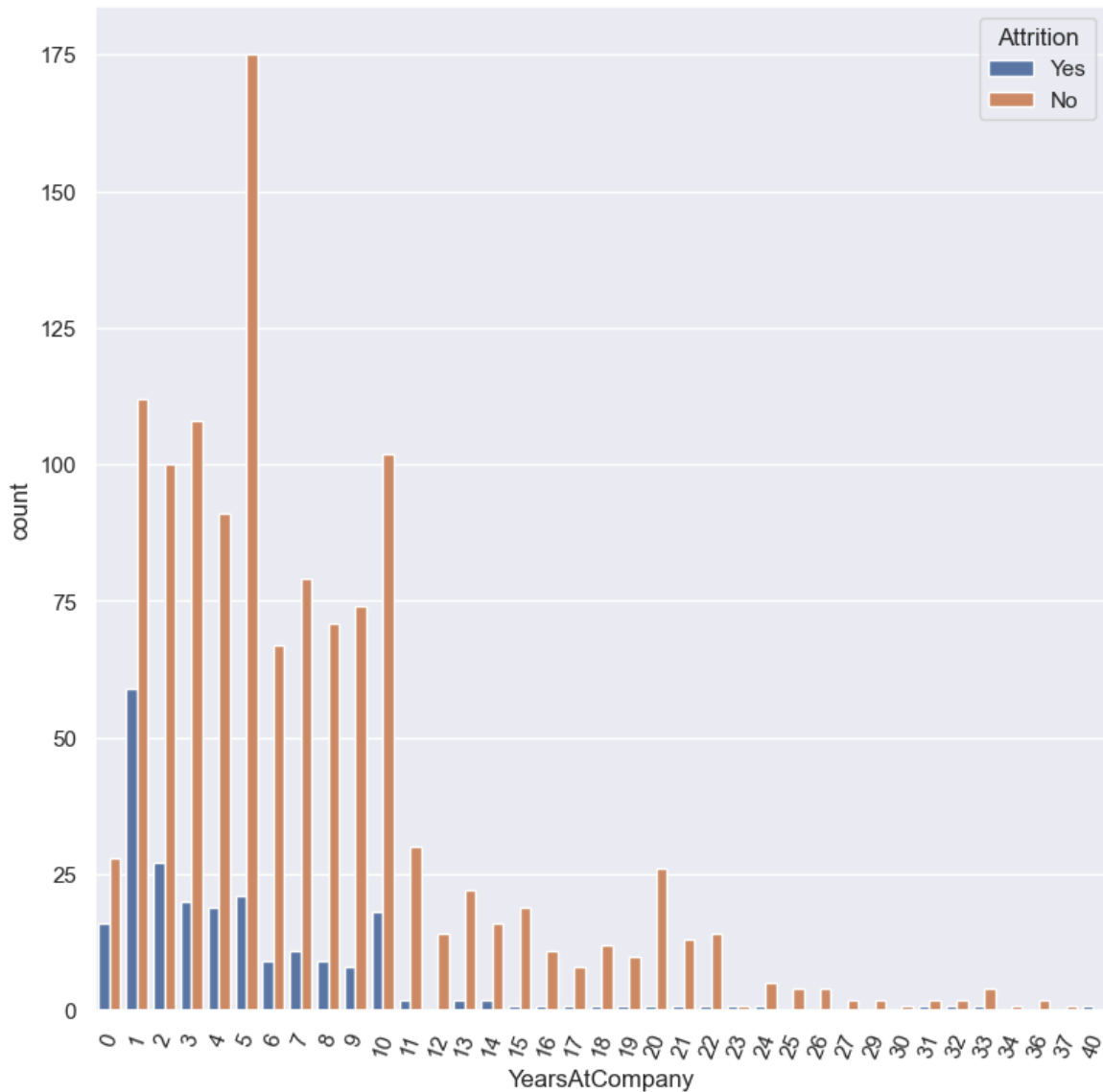
9. Biểu đồ phân bố số năm làm việc tại công ty theo trạng thái nghỉ việc

```
plt.figure(figsize=(9,9))
sns.countplot(x=df['YearsAtCompany'],hue =df['Attrition'])
plt.xticks(rotation = 70)
plt.show()
```

Hình 3.28: Mã nguồn biểu đồ số năm làm việc tại công ty theo trạng thái nghỉ việc

- `plt.figure(figsize=(9,9))` thiết lập kích thước khung hình vuông để hiển thị rõ các cột dữ liệu trải dài
- Sử dụng hàm `sns.countplot` để thống kê số lượng nhân viên tương ứng với số năm làm việc tại công ty (`YearsAtCompany`)
- Tham số `hue=df['Attrition']` giúp phân tách mỗi mốc thâm niên thành 2 cột (Nghỉ việc/O lại) để so sánh tỷ lệ rời đi ở từng giai đoạn (ví dụ: nhân viên mới hay nhân viên lâu năm dễ nghỉ hơn)

- `plt.xticks(rotation = 70)` xoay các nhãn trên trục hoành một góc 70 độ để tránh hiện tượng các con số bị chồng chéo lên nhau do có nhiều mốc thời gian
- `plt.show()` hiển thị biểu đồ hoàn chỉnh lên màn hình



Hình 3.29: Biểu đồ số năm làm việc tại công ty theo trạng thái nghỉ việc

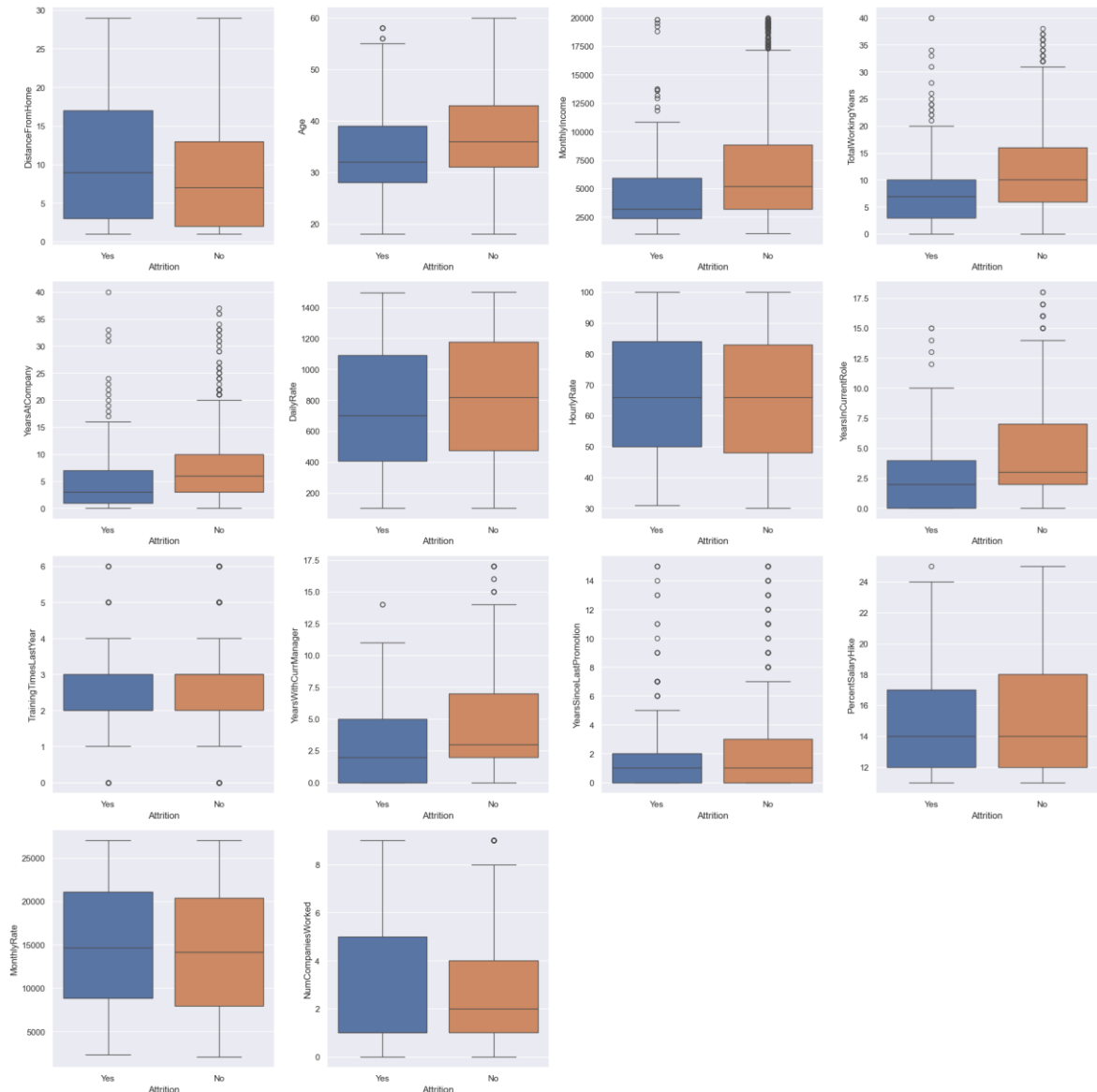
10. Biểu đồ Boxplot cho từng cột theo trạng thái nghỉ việc

```
fig, axs = plt.subplots(4, 4, figsize=(20,20))
sns.boxplot(y='DistanceFromHome',x = 'Attrition', hue = 'Attrition',data = df, ax=axs[0][0])
sns.boxplot(y='Age',x = 'Attrition', hue = 'Attrition',data = df , ax=axs[0][1])
sns.boxplot(y='MonthlyIncome',x = 'Attrition', hue = 'Attrition',data = df, ax=axs[0][2])
sns.boxplot(y='TotalWorkingYears',x = 'Attrition', hue = 'Attrition',data = df, ax=axs[0][3])
sns.boxplot(y='YearsAtCompany',x = 'Attrition', hue = 'Attrition',data = df, ax=axs[1][0])
sns.boxplot(y='DailyRate',x = 'Attrition', hue = 'Attrition',data = df, ax=axs[1][1])
sns.boxplot(y='HourlyRate',x = 'Attrition', hue = 'Attrition',data = df, ax=axs[1][2])
sns.boxplot(y='YearsInCurrentRole',x = 'Attrition', hue = 'Attrition',data = df, ax=axs[1][3])
sns.boxplot(y='TrainingTimesLastYear',x = 'Attrition', hue = 'Attrition',data = df, ax=axs[2][0])
sns.boxplot(y='YearsWithCurrManager',x = 'Attrition', hue = 'Attrition',data = df, ax=axs[2][1])
sns.boxplot(y='YearsSinceLastPromotion',x = 'Attrition', hue = 'Attrition',data = df, ax=axs[2][2])
sns.boxplot(y='PercentSalaryHike',x = 'Attrition', hue = 'Attrition',data = df, ax=axs[2][3])
sns.boxplot(y='MonthlyRate',x = 'Attrition', hue = 'Attrition',data = df, ax=axs[3][0])
sns.boxplot(y='NumCompaniesWorked',x = 'Attrition', hue = 'Attrition',data = df, ax=axs[3][1])
fig.delaxes(axs[3][2])
fig.delaxes(axs[3][3])
plt.tight_layout()
plt.show()
```

Hình 3.30: Mã nguồn biểu đồ boxplot cho từng cột theo trạng thái nghỉ việc

- `fig, axs = plt.subplots(4, 4, figsize=(20,20))` khởi tạo một khung hình lớn dạng lưới kích thước 4x4 (tạo ra 16 vị trí vẽ biểu đồ con) với kích thước tổng thể 20x20 inch
- Sử dụng hàm `sns.boxplot` lặp lại nhiều lần để vẽ biểu đồ hộp cho từng biến định lượng (như Tuổi, Lương, Thâm niên...)
- Tham số `y='Tên_cột'` thay đổi tương ứng với từng biến số cần phân tích (ví dụ: `DistanceFromHome`, `MonthlyIncome`...)
- `x='Attrition'` và `hue='Attrition'` giúp so sánh trực quan sự khác biệt về trung vị (median) và khoảng tứ phân vị (IQR) giữa nhóm Nghỉ việc và Ở lại
- `ax=axs[hàng][cột]` chỉ định vị trí cụ thể của từng biểu đồ nhỏ trong lưới 4x4
- `fig.delaxes(axs[3][2])` và `fig.delaxes(axs[3][3])` thực hiện xóa bỏ 2 ô trống cuối cùng ở hàng 4 (vì chỉ vẽ 14 biểu đồ trên lưới 16 ô) giúp giao diện sạch đẹp hơn
- `plt.tight_layout()` tự động căn chỉnh khoảng cách lề giữa các biểu đồ con để tiêu đề và trục không bị chồng lấn lên nhau

- `plt.show()` hiển thị toàn bộ hệ thống biểu đồ hoàn chỉnh lên màn hình



Hình 3.31: Biểu đồ boxplot cho từng cột theo trạng thái nghỉ việc

3.4 Áp dụng Model

- Chuẩn hóa
 - Chuẩn hóa là quá trình đưa các thuộc tính dữ liệu có các đơn vị và khoảng giá trị khác nhau về cùng một hệ quy chiếu. Tại sao cần chuẩn hóa :
 - Tránh sự chênh lệch đơn vị giữa giá trị nhỏ nhất và lớn nhất
 - Tăng sự hội tụ giúp hàm tìm kết quả nhanh hơn
 - Giúp công bằng giữa các đặc trưng

```
continuous_cols = [
    'Age', 'DailyRate', 'DistanceFromHome', 'HourlyRate',
    'MonthlyIncome', 'MonthlyRate', 'TotalWorkingYears',
    'PercentSalaryHike', 'YearsAtCompany',
    'YearsInCurrentRole', 'YearsSinceLastPromotion', 'YearsWithCurrManager',
    'NumCompaniesWorked', 'TrainingTimesLastYear'
]
scaler = StandardScaler()
df_continuous = pd.DataFrame(scaler.fit_transform(df[continuous_cols]), columns=continuous_cols)
df_continuous.head()
```

	0	1	2	3	4
Age	0.446350	1.322365	0.008343	-0.429664	-1.086676
DailyRate	0.742527	-1.297775	1.414363	1.461466	-0.524295
DistanceFromHome	-1.010909	-0.147150	-0.887515	-0.764121	-0.887515
HourlyRate	1.383138	-0.240677	1.284725	-0.486709	-1.274014
MonthlyIncome	-0.108350	-0.291719	-0.937654	-0.763634	-0.644858
MonthlyRate	0.726020	1.488876	-1.674841	1.243211	0.325900
TotalWorkingYears	-0.421642	-0.164511	-0.550208	-0.421642	-0.678774
PercentSalaryHike	-1.150554	2.129306	-0.057267	-1.150554	-0.877232
YearsAtCompany	-0.164613	0.488508	-1.144294	0.161947	-0.817734
YearsInCurrentRole	-0.063296	0.764998	-1.167687	0.764998	-0.615492
YearsSinceLastPromotion	-0.679146	-0.368715	-0.679146	0.252146	-0.058285
YearsWithCurrManager	0.245834	0.806541	-1.155935	-1.155935	-0.595227
NumCompaniesWorked	2.125136	-0.678049	1.324226	-0.678049	2.525591
TrainingTimesLastYear	-2.171982	0.155707	0.155707	0.155707	0.155707

Hình 3.32: Chuẩn hóa dữ liệu

- Mã hóa dữ liệu
 - Mã hóa dữ liệu là quá trình chuyển đổi các dữ liệu từ dạng văn bản (chuỗi ký tự/định danh) sang dạng số học để máy tính có thể hiểu và xử lý. Tại sao cần mã hóa:

- Máy tính và các thuật toán học máy (như Hồi quy, Mạng nơ-ron) chỉ hoạt động dựa trên các phép toán số học, không thể tính toán trực tiếp trên văn bản
- Giúp chuyển đổi các thông tin định tính (như Giới tính, Phòng ban) thành dữ liệu định lượng đầu vào cho mô hình
- Tối ưu hóa hiệu suất tính toán và lưu trữ của hệ thống so với việc xử lý dữ liệu thô dạng chuỗi

```
cat_vars = ['Education', 'EnvironmentSatisfaction', 'JobInvolvement',  
            'JobSatisfaction', 'PerformanceRating', 'RelationshipSatisfaction',  
            'WorkLifeBalance', 'StockOptionLevel', 'JobLevel', 'BusinessTravel', 'Department', 'EducationField',  
            'JobRole', 'MaritalStatus', 'Gender', 'OverTime']  
le = LabelEncoder()  
df_cat = df[cat_vars].apply(le.fit_transform)  
df_cat.head().T
```

	0	1	2	3	4
Education	1	0	1	3	0
EnvironmentSatisfaction	1	2	3	3	0
JobInvolvement	2	1	1	2	2
JobSatisfaction	3	1	2	2	1
PerformanceRating	0	1	0	0	0
RelationshipSatisfaction	0	3	1	2	3
WorkLifeBalance	0	2	2	2	2
StockOptionLevel	0	1	0	0	1
JobLevel	1	1	0	0	0
BusinessTravel	2	1	2	1	2
Department	2	1	1	1	1
EducationField	1	1	4	1	3
JobRole	7	6	2	6	2
MaritalStatus	2	1	2	1	1
Gender	0	1	1	0	1
OverTime	1	0	1	1	0

Hình 3.33: Mã hóa dữ liệu

- Chia dữ liệu train và test
- Chia dữ liệu dữ liệu thành tập huấn luyện (train) và tập kiểm tra (test):
- X là các đặc trưng (feature) và y là biến mục tiêu (target)
- Test_size=0.2 là lấy 20% dữ liệu sử dụng cho tập kiểm tra còn 80% dữ liệu còn lại sẽ được sử dụng cho tập huấn luyện

- `Random_state = 42` đảm bảo rằng quá trình chia dữ liệu sẽ được lặp lại một cách nhất quán mỗi lần chạy mã, giúp ta có thể tái tạo lại kết quả

```
X = df1.drop('Attrition', axis=1)
y = df1['Attrition'].map({'Yes': 1, 'No': 0})

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

Hình 3.34: Mã nguồn chia 2 tập train và test

```
print(X_train.shape[0])
print(X_test.shape[0])
print(y_train.value_counts())
print(f"Tỉ lệ nghỉ việc trên train: {y_train.mean():.2f}, test: {y_test.mean():.2f}")
```

1176
294
Attrition
0 986
1 190
Name: count, dtype: int64
Tỉ lệ nghỉ việc trên train: 0.16, test: 0.16

Hình 3.35: Kết quả sau khi chia dữ liệu

- Xử lý dữ liệu mất cân bằng

SMOTE (Synthetic Minority Over-sampling Technique) là một công cụ mạnh mẽ giúp khắc phục tình trạng mất cân bằng dữ liệu. Trong bài toán quản trị nhân sự thực tế, số lượng nhân viên "Ở lại" (No) thường lớn hơn rất nhiều so với số lượng nhân viên "Nghỉ việc" (Yes). Nếu mô hình huấn luyện trên dữ liệu chênh lệch như vậy, nó sẽ bị thiên kiến về phía nhóm đa số và dự đoán kém chính xác đối với nhóm nhân viên có nguy cơ rời đi. SMOTE giải quyết vấn đề này bằng cách tạo ra các mẫu "tổng hợp" (dữ liệu nhân tạo dựa trên đặc trưng thực tế) cho nhóm thiểu số để cân bằng lại cán cân dữ liệu.

Sử dụng tham số `random_state` (ví dụ = 42 hoặc = 2) để cố định quy trình sinh dữ liệu ngẫu nhiên, đảm bảo kết quả cân bằng mẫu sẽ được lặp lại một cách nhất quán và chính xác sau mỗi lần chạy mã.

```
smote = SMOTE()
X_res, y_res = smote.fit_resample(X_train, y_train)
print(y_res.value_counts())
```

```
Attrition
0      986
1      986
Name: count, dtype: int64
```

Hình 3.36: Xử lý mất cân bằng dữ liệu bằng SMOTE

- Huấn luyện mô hình

```
def evaluate_clf(model,model_name):
    y_pred = model.predict(X_test)
    y_test_proba = model.predict_proba(X_test)

    accuracy = accuracy_score(y_test,y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    auc = roc_auc_score(y_test, y_test_proba[:,1])

    print('Accuracy:', accuracy_score(y_test, y_pred))
    print('Classification Report:\n', classification_report(y_test, y_pred))

    fig, axs = plt.subplots(1,3)
    fig.set_size_inches(15,4)
    plt.tight_layout()
    PrecisionRecallDisplay.from_estimator(model, X_test, y_test, ax=axs[0])
    RocCurveDisplay.from_estimator(model, X_test, y_test, ax=axs[1])
    ConfusionMatrixDisplay.from_estimator(model, X_test, y_test, ax=axs[2])
    axs[0].set_title('Precision-Recall curve')
    axs[1].set_title('ROC curve')

    eval_dict = {'modelname':model_name,'accuracy':accuracy,
                'precision':precision,'recall':recall,'f1':f1,'roc_auc':auc}
    return eval_dict

model_metrics = pd.DataFrame(columns=['modelname', 'accuracy', 'precision','recall','f1', 'roc_auc'])
```

Hình 3.37: Mã nguồn huấn luyện mô hình

- Xây dựng hàm `evaluate_clf` để tự động hóa quy trình đánh giá và trực quan hóa kết quả của các mô hình phân loại (như Logistic Regression, Random Forest)
- Thực hiện dự đoán nhãn lớp bằng `model.predict` và dự đoán xác suất bằng `model.predict_proba` (cần thiết để tính chỉ số AUC)

- Tính toán 5 chỉ số đo lường quan trọng nhất: Accuracy (Độ chính xác), Precision (Độ chuẩn xác), Recall (Độ nhạy), F1-Score và ROC-AUC
- In ra màn hình báo cáo chi tiết bằng `classification_report` để người dùng có cái nhìn tổng quan ngay lập tức
- Sử dụng `plt.subplots(1,3)` để vẽ 3 biểu đồ đánh giá nằm ngang hàng nhau giúp so sánh trực quan:
- Precision-Recall Curve: Đánh giá sự đánh đổi giữa độ chuẩn xác và độ nhạy (quan trọng với dữ liệu mất cân bằng)
- ROC Curve: Đánh giá khả năng phân loại của mô hình ở các ngưỡng xác suất khác nhau
- Confusion Matrix: Hiển thị số lượng mẫu dự đoán đúng/sai thực tế (True Positive, False Negative...)
- Đóng gói kết quả vào từ điển `eval_dict` và trả về để lưu trữ
- Khởi tạo DataFrame rộng `model_metrics` dùng để bảng hóa và so sánh hiệu năng giữa các thuật toán khác nhau sau khi chạy xong

3.4.1 Model Logistic Regression

Trước cân bằng dữ liệu

```
model = LogisticRegression()  
model.fit(X_train, y_train)  
metrics = evaluate_clf(model, 'Logistic regression')
```

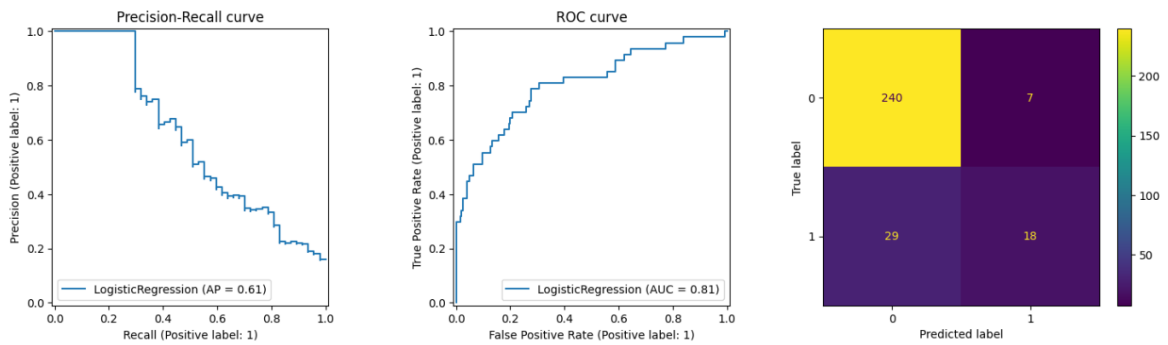
Accuracy: 0.8775510204081632

Classification Report:

	precision	recall	f1-score	support
0	0.89	0.97	0.93	247
1	0.72	0.38	0.50	47
accuracy			0.88	294
macro avg	0.81	0.68	0.72	294
weighted avg	0.86	0.88	0.86	294

Hình 3.38: Model logistic regression trước smote

- Khởi tạo và thực hiện quá trình huấn luyện cho thuật toán Logistic Regression
- `model = LogisticRegression()` tạo ra một đối tượng mô hình rỗng, chuẩn bị cho việc học máy
- Hàm `.fit(X_train, y_train)` là bước quan trọng nhất, tại đây mô hình sẽ phân tích dữ liệu huấn luyện để tìm ra các hệ số (trọng số) tối ưu nhằm vẽ ra đường ranh giới phân loại giữa nhóm Nghỉ việc và Ở lại
- Gọi hàm `evaluate_clf` (đã định nghĩa trước đó) để tự động hóa việc kiểm tra hiệu quả của mô hình
- Truyền mô hình đã học (`model`) và tên hiển thị 'Logistic regression' vào hàm để tính toán các chỉ số đo lường và vẽ biểu đồ đánh giá (ROC, Confusion Matrix)
- Kết quả trả về được lưu vào biến `metrics`, giúp lưu trữ các con số hiệu năng (Accuracy, F1-score...) để dễ dàng so sánh với các thuật toán khác sau này



Hình 3.39: Biểu đồ Precision-Recall, ROC và Predicted label của model logistic regression trước smote

Sau cân bằng dữ liệu

```
lr = LogisticRegression()
lr.fit(X_res, y_res)
metrics = evaluate_clf(lr, 'Logistic regression')
model_metrics.loc[len(model_metrics)] = metrics
```

Accuracy: 0.7789115646258503

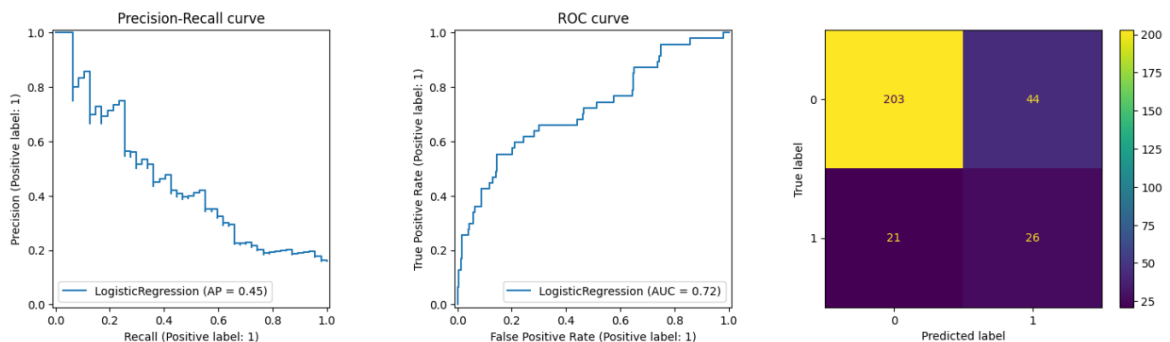
Classification Report:

	precision	recall	f1-score	support
0	0.91	0.82	0.86	247
1	0.37	0.55	0.44	47
accuracy			0.78	294
macro avg	0.64	0.69	0.65	294
weighted avg	0.82	0.78	0.80	294

Hình 3.40: Model logistic regression sau smote

- Khởi tạo và huấn luyện mô hình **Logistic Regression** trên tập dữ liệu đã qua xử lý cân bằng mẫu
- Hàm `.fit(X_res, y_res)` thực hiện quá trình học, điểm quan trọng ở đây là sử dụng `X_res` và `y_res` (dữ liệu đã cân bằng bằng SMOTE) thay vì dữ liệu gốc, giúp mô hình không bị thiên vị nhóm đa số
- Gọi hàm `evaluate_clf` để đánh giá toàn diện hiệu suất của mô hình

- Tính toán các chỉ số (Accuracy, Precision, Recall...) và trả về biến metrics
- Cập nhật kết quả vào bảng tổng hợp model_metrics để so sánh
- Câu lệnh model_metrics.loc[len(model_metrics)] = metrics là kỹ thuật thêm một dòng dữ liệu mới vào vị trí cuối cùng của DataFrame, giúp lưu lại lịch sử kết quả để dễ dàng so sánh với các thuật toán khác sau này



Hình 3.41: Biểu đồ Precision-Recall, ROC và Predicted lable của model logistic regression sau smote

3.4.2 Model Random Forest

```
rf = RandomForestClassifier()
rf.fit(X_res, y_res)
metrics = evaluate_clf(rf, 'Random Forest')
model_metrics = pd.concat([model_metrics, pd.DataFrame([metrics])], ignore_index=True)
```

Accuracy: 0.826530612244898

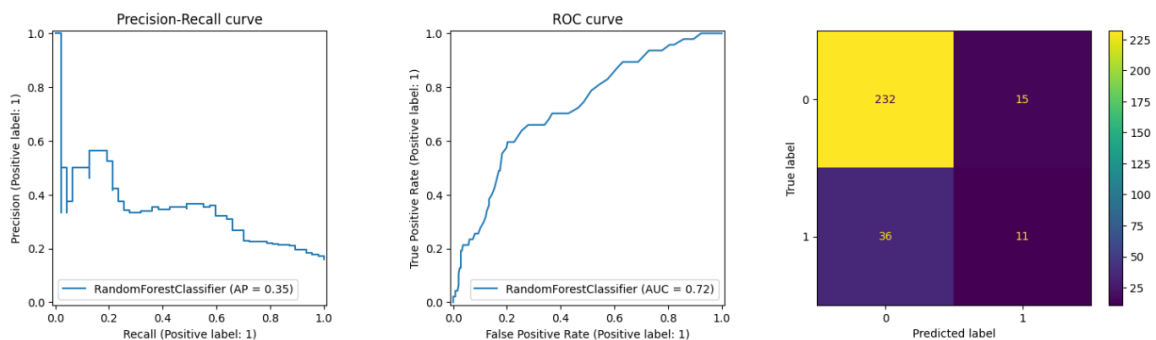
Classification Report:

	precision	recall	f1-score	support
0	0.87	0.94	0.90	247
1	0.42	0.23	0.30	47
accuracy			0.83	294
macro avg	0.64	0.59	0.60	294
weighted avg	0.79	0.83	0.81	294

Hình 3.42: Model random forest

- Khởi tạo và huấn luyện mô hình **Random Forest** (Rừng ngẫu nhiên) trên tập dữ liệu đã được cân bằng

- RandomForestClassifier() tạo ra một mô hình tổ hợp bao gồm nhiều cây quyết định hoạt động cùng nhau để đưa ra dự đoán chính xác hơn và giảm thiểu rủi ro overfitting
- Hàm .fit(X_res, y_res) huấn luyện mô hình dựa trên dữ liệu đã xử lý SMOTE, giúp cải thiện đáng kể khả năng nhận diện nhóm nhân viên nghỉ việc (nhóm thiểu số)
- Đánh giá và lưu trữ kết quả để so sánh hiệu năng với mô hình trước đó
- Gọi hàm evaluate_clf để lấy các chỉ số đo lường (như Accuracy, F1-Score) và vẽ biểu đồ đánh giá chuyên sâu
- pd.concat thực hiện ghép nối dòng kết quả mới (metrics) vào bảng tổng hợp model_metrics, phục vụ cho việc so sánh cuối cùng xem Random Forest hay Logistic Regression hiệu quả hơn



Hình 3.43: Biểu đồ Precision-Recall, ROC và Predicted lable của model random forest

3.4.3 So sánh và đánh giá mô hình

```
model_metrics = model_metrics.set_index('modelname')
model_metrics
```

	accuracy	precision	recall	f1	roc_auc
modelname					
Logistic regression	0.778912	0.371429	0.553191	0.444444	0.718925
Random Forest	0.826531	0.423077	0.234043	0.301370	0.723663

Hình 3.44: So sánh hai mô hình

- Mô hình Logistic Regression:

- Ưu điểm: Có chỉ số Recall cao hơn đáng kể (0.553) so với Random Forest. Trong bài toán quản trị nhân sự, điều này rất quan trọng vì nó có nghĩa là mô hình tìm được hơn 55% số nhân viên thực sự có ý định nghỉ việc.
- Nhược điểm: Precision thấp (0.371), nghĩa là mô hình này bị "nhảy cảm", dẫn đến việc dự đoán nhầm nhiều nhân viên đang hài lòng là sẽ nghỉ việc (báo động giả).

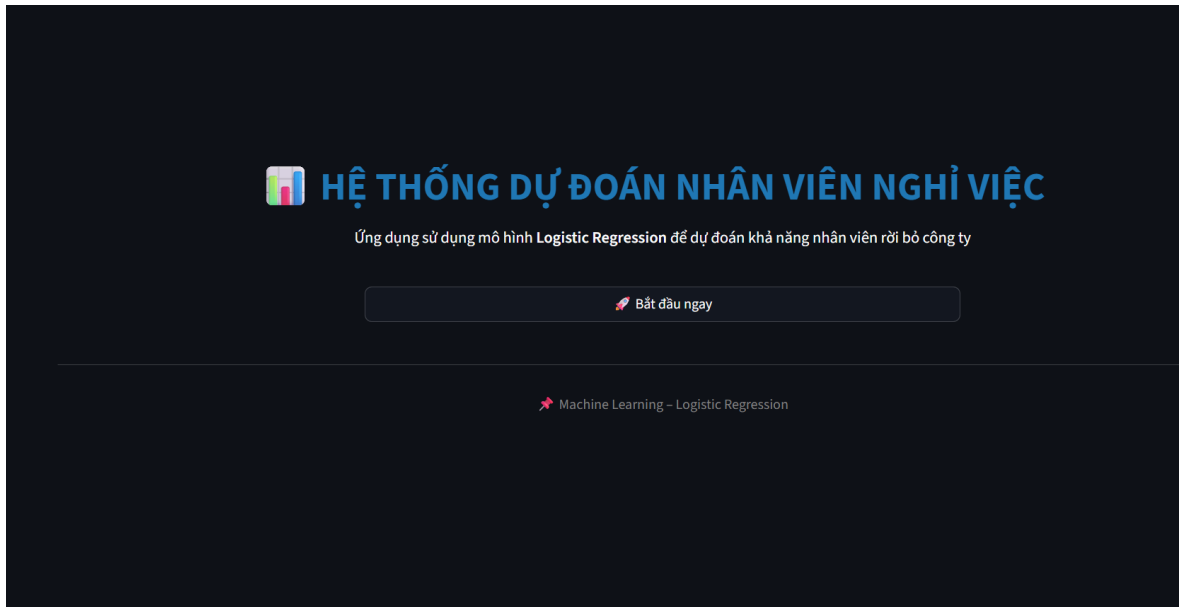
- Mô hình Random Forest:

- Ưu điểm: Độ chính xác tổng thể (Accuracy) cao hơn (0.826 so với 0.778). Nó hoạt động ổn định với đa số dữ liệu (nhóm nhân viên ở lại).
- Nhược điểm: Recall quá thấp (0.234). Mô hình này bỏ sót tới gần 77% số nhân viên nghỉ việc thực tế. Đây là một điểm yếu chí mạng nếu mục tiêu là giữ chân nhân tài.

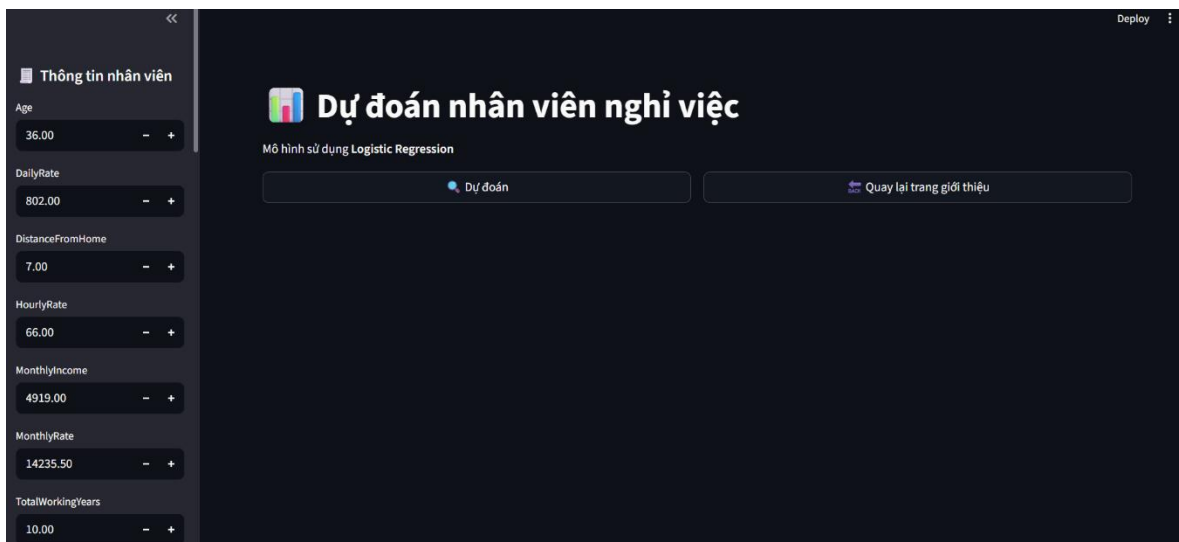
Tuy nhiên trong thực tế quản trị, việc bỏ sót nhân viên có nguy cơ nghỉ việc (False Negative) gây thiệt hại lớn cho công ty về chi phí tuyển dụng và đào tạo lại. Trong Logistic Regression có điểm Recall khoảng 55%, trong khi đó Random Forest chỉ khoảng 23% (tức là nó bỏ sót phần lớn nhân viên cần quan tâm). Vậy theo em,

nên ưu tiên sử dụng model Logistic Regression để thực hiện dự đoán, giúp bộ phận HR chủ động tiếp cận được nhiều nhân viên có nguy cơ rời đi hơn.

3.5 Sản phẩm



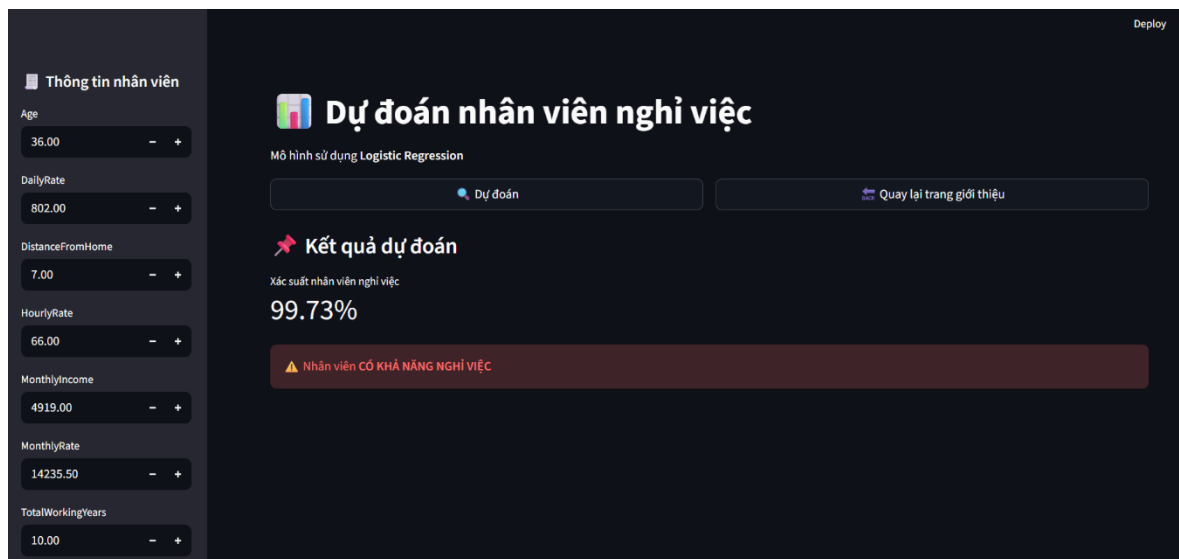
Hình 3.45: Tiêu đề sản phẩm



Hình 3.46: Giao diện sản phẩm

Phần này được sử dụng để dự đoán nguy cơ nhân viên nghỉ việc dựa trên các dữ liệu được nhập vào hệ thống. Các thông tin đầu vào bao gồm thông tin cá nhân (như Tuổi), đặc điểm công việc (như Thâm niên, Khoảng cách từ nhà) và mức độ đãi ngộ

(như Lương tháng, Lương theo ngày/giờ) để mô hình phân tích và đưa ra kết quả cảnh báo.



Hình 3.47: Kết quả dự đoán bằng model logistic regression

KẾT LUẬN

Bài tập lớn môn Học máy cơ bản đã hoàn thành trọn vẹn với việc hệ thống hóa cơ sở lý thuyết về quy trình Khai phá dữ liệu (Data Mining), các kỹ thuật tiền xử lý và hai thuật toán phân loại nòng cốt: Logistic Regression và Random Forest. Đề tài đã vận dụng thành công các kiến thức này để giải quyết bài toán dự báo nhân sự thực tế với những kết quả đáng ghi nhận:

Về xử lý dữ liệu: Đã thực hiện hiệu quả quy trình làm sạch và chuẩn hóa dữ liệu. Đặc biệt, việc áp dụng kỹ thuật cân bằng dữ liệu đã giúp khắc phục tình trạng chênh lệch mẫu, tạo tiền đề quan trọng để mô hình học được chính xác các đặc trưng của nhóm nhân viên có nguy cơ nghỉ việc.

Về hiệu năng mô hình: Kết quả thực nghiệm và so sánh cho thấy thuật toán Logistic Regression hoạt động hiệu quả và vượt trội hơn so với Random Forest trên bộ dữ liệu này. Logistic Regression không chỉ mang lại độ chính xác cao, ổn định mà còn thể hiện ưu thế về khả năng tổng quát hóa (tránh hiện tượng Overfitting thường gặp ở các mô hình cây quyết định). Ngoài ra, mô hình này còn giúp dễ dàng giải thích mức độ ảnh hưởng của từng yếu tố (như thu nhập, khoảng cách nhà...) đến quyết định nghỉ việc thông qua các trọng số hồi quy.

Về tính ứng dụng thực tiễn: Đề tài đã hiện thực hóa mô hình thành sản phẩm phần mềm demo, cho phép người dùng nhập liệu và nhận kết quả cảnh báo nguy cơ nghỉ việc tức thì. Đây là bước chuyển đổi quan trọng từ kết quả nghiên cứu sang công cụ hỗ trợ ra quyết định (Decision Support System) hữu ích cho doanh nghiệp.

TÀI LIỆU THAM KHẢO

- 1) Swinburne V.
- 2) Youtube: học lập trình cùng Phát: Machine learning cơ bản Logistic Regression
- 3) Numpy : [Giới thiệu về Numpy - Nguyen Van Hoang \(viblo.asia\)](https://viblo.asia/gioi-thieu-ve-numpy)
- 4) N.V . Hậu, P. M. Chuẩn, và N. V. Quyết , “*Giáo trình Học máy cơ bản*”, Nhà xuất bản Khoa học và Kỹ thuật, 2022
- 5) GeekforGeeks: random forest algorithm in machine learning